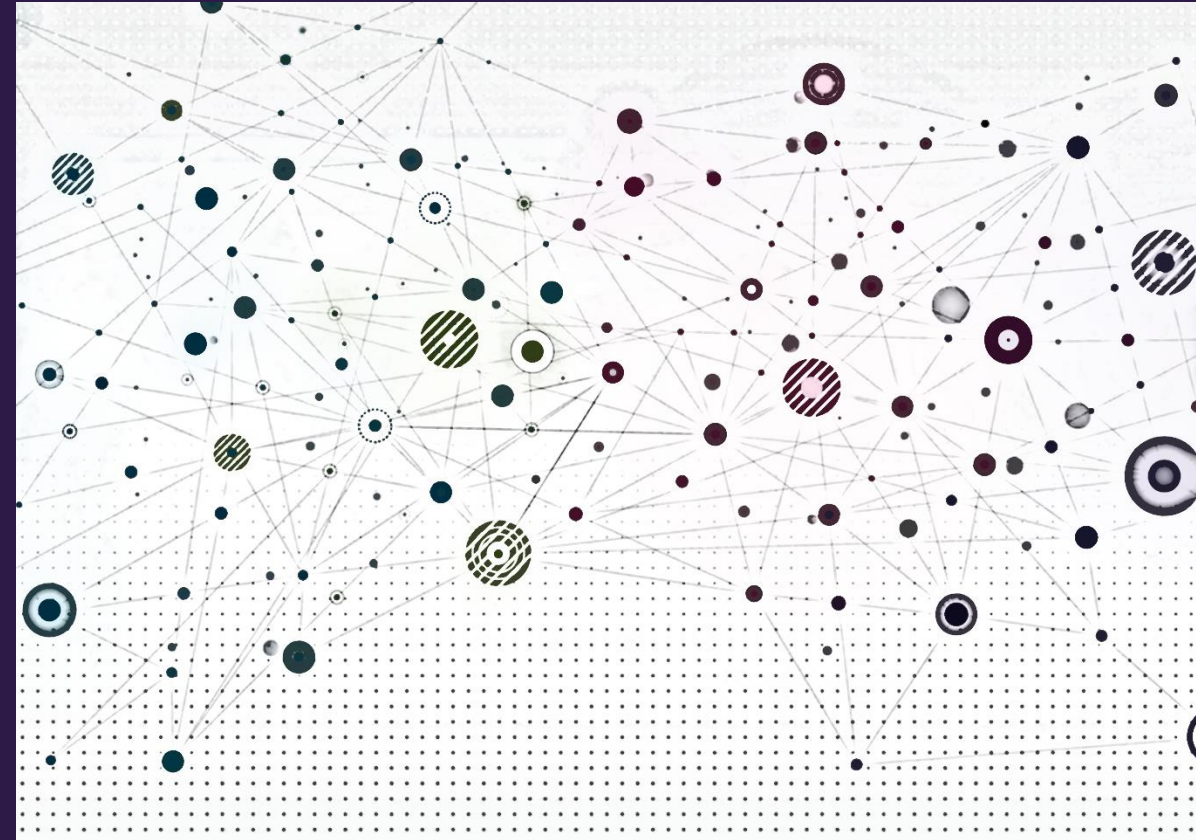


# XAI for GNNs





# Overview

- What is explainable AI?
- Motivation for explainable AI (XAI)
- XAI methods for tabular, image, text data
- XAI methods for graphs
- XAI in PyG



# What is explainable AI (XAI)?





# What is explainability?

- “... *the ability to explain or to present in understandable terms to a human*” (Doshi-Velez & Kim 2017)
- “*descriptions are diverse ... refers to more than one concept*” (Lipton 2016)
- There is no single definition
- Attempts to quantify explainability require subjective decisions on desiderata and measurement



# What is eXplainable Artificial Intelligence (XAI)

- An emerging (or maybe resurgent) research area in AI
- Goal is to develop knowledge and methods that can be used to “explain”:
  - How AI systems arrive at specific solutions
  - Which features most influenced a given output
  - Which and how features interact
  - Operating characteristics
  - Model biases
  - Accuracy relative to input regions
  - How the model learned



# What is explainable AI?

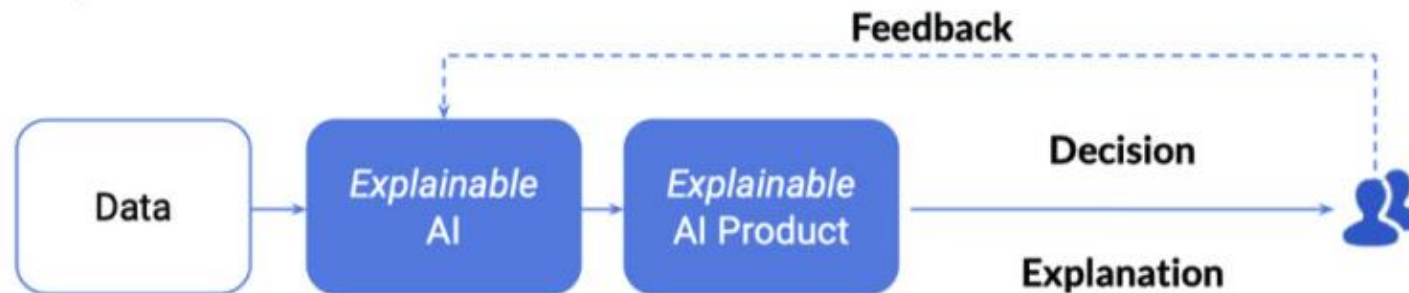
## Black Box AI



## Confusion with Today's AI Black Box

- Why did you do that?
- Why did you not do that?
- When do you succeed or fail?
- How do I correct an error?

## Explainable AI



## Clear & Transparent Predictions

- I understand why
- I understand why not
- I know why you succeed or fail
- I understand, so I trust you



# What makes an AI explainable?

No consensus definition but most researchers agree explainability is a construct encompassing several concepts including:

- *Interpretability* – degree to which a human can infer the cause of and predict a model output
- *Understandability* – degree to which a user can determine how the system works
- *Usability* – the ease with which a user can operate the AI
- *Utility* – the practical usefulness of the AI

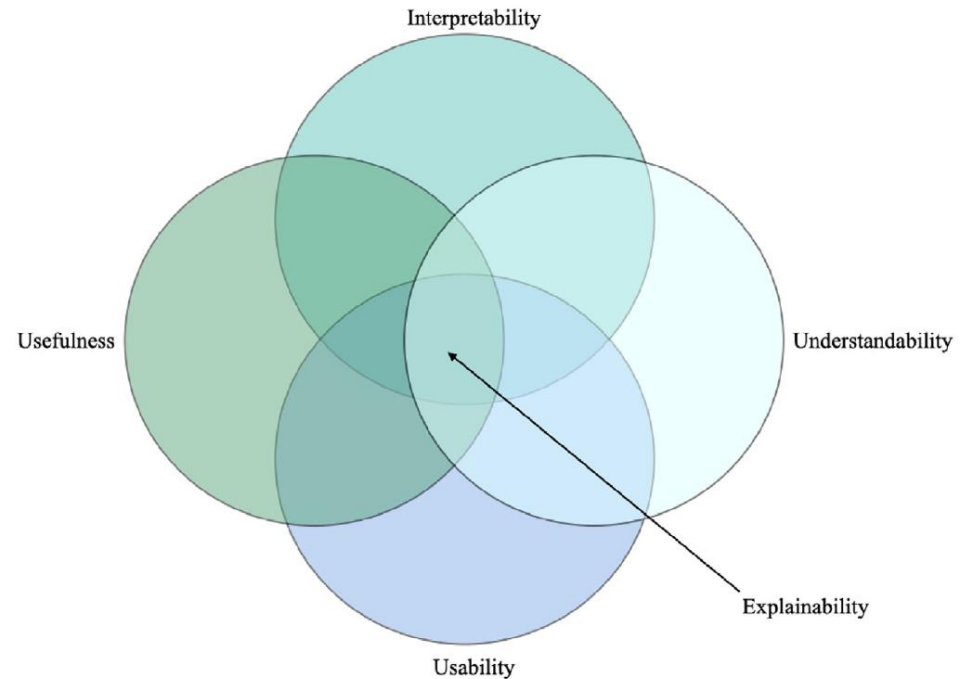


Figure credit: Combi, Carlo, et al. "A manifesto on explainability for artificial intelligence in medicine." *Artificial Intelligence in Medicine* 133 (2022): 102423.



# Why do we need special methods for explainability?

Arises when learning objectives fail to capture other real-world desiderata

Prediction Loss

$$L(y, \hat{y}) + G(\lambda, \hat{\Theta}) + ?$$

Regularization

## Objective Function Limitations

- Difficult to formalize many real-world desiderata in mathematical form
- For example, how would one form an objective to encode "trust" or "fairness"



# Motivation for XAI





# Auditing for high-stakes decision

Many real-world applications can lead to biased decisions that potentially cause serious harm

- Criminal justice

# Auditing for high-stakes decision

Many real-world applications can lead to biased decisions that potentially cause serious harm

## Criminal justice



### How We Analyzed the COMPAS Recidivism Algorithm

by Jeff Larson, Surya Mattu, Lauren Kirchner and Julia Angwin  
May 23, 2016

Only 20% of people considered at risk of recidivism ended committing a new crime

The New York Times

Opinion

OP-ED CONTRIBUTOR

### When a Computer Program Keeps You in Jail

By Rebecca Wexler

June 13, 2017

# Auditing for high-stakes decision

Many real-world applications can lead to biased decisions that potentially cause serious harm

## Finance

Review Article | [Open access](#) | Published: 22 April 2025

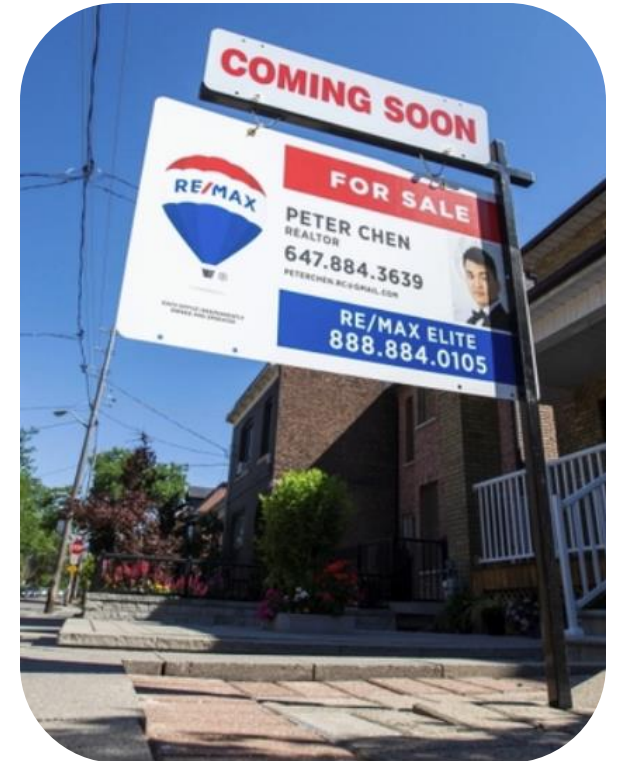
### AI integration in financial services: a systematic review of trends and regulatory challenges

[Darko B. Vuković](#) ✉, [Senanu Dekpo-Adza](#) & [Stefana Matović](#)

### Bias in Code: Algorithm Discrimination in Financial Systems

BY SPENCER WANG

JANUARY 28, 2025



# Auditing for high-stakes decision

Many real-world applications can lead to biased decisions that potentially cause serious harm

## Healthcare

Yale SCHOOL OF MEDICINE

### ‘Bias in, bias out’: Tackling bias in medical artificial intelligence

By Saul Asiel Flores

November 18, 2024

Los Angeles, Jun 30, 2025

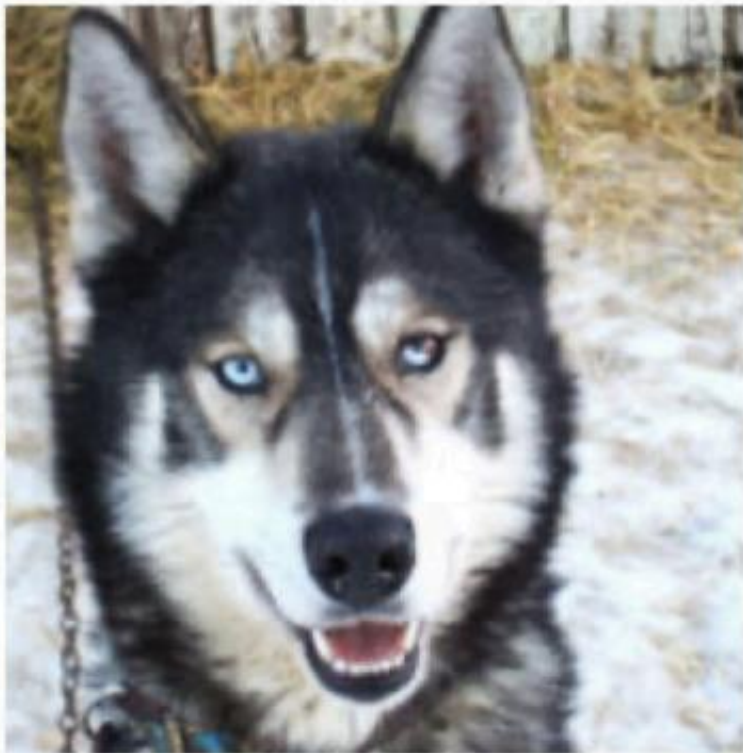
### Cedars-Sinai Study Shows Racial Bias in AI-Generated Treatment Regimens for Psychiatric Patients



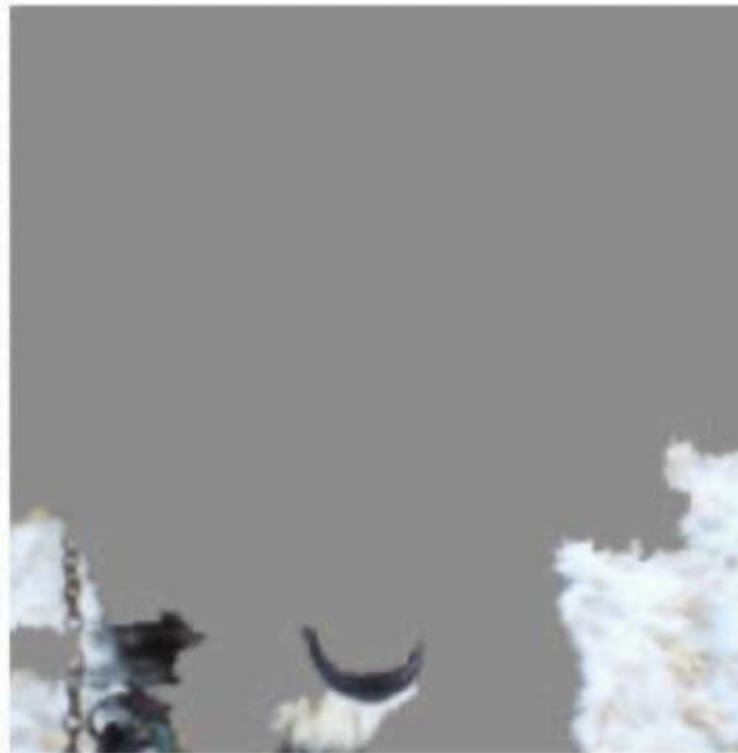


# Model error analysis

Explanations can help identify patterns in model errors



(a) Husky classified as wolf



(b) Explanation

Background bias: Model predictions of object classification can be influenced by the background. Here, the model predicts wolf because of snow in the background.



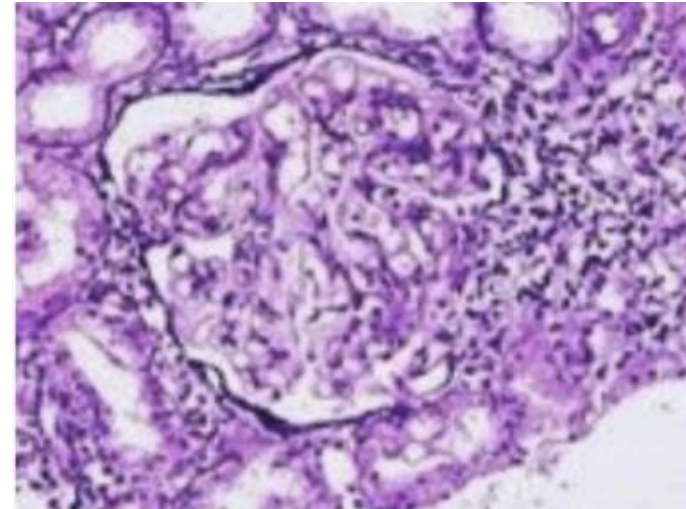
# Model error analysis

Explanations can support model verification and risks

Which scenarios pose risks to autonomous vehicle systems?



Under what conditions might AI misdiagnose a disease?

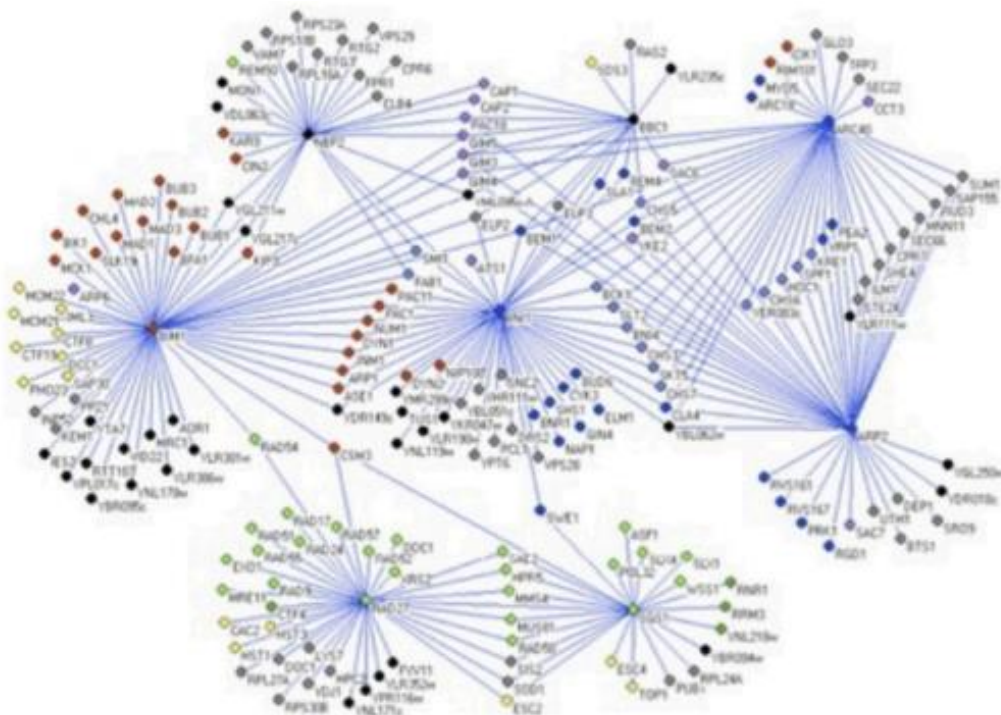




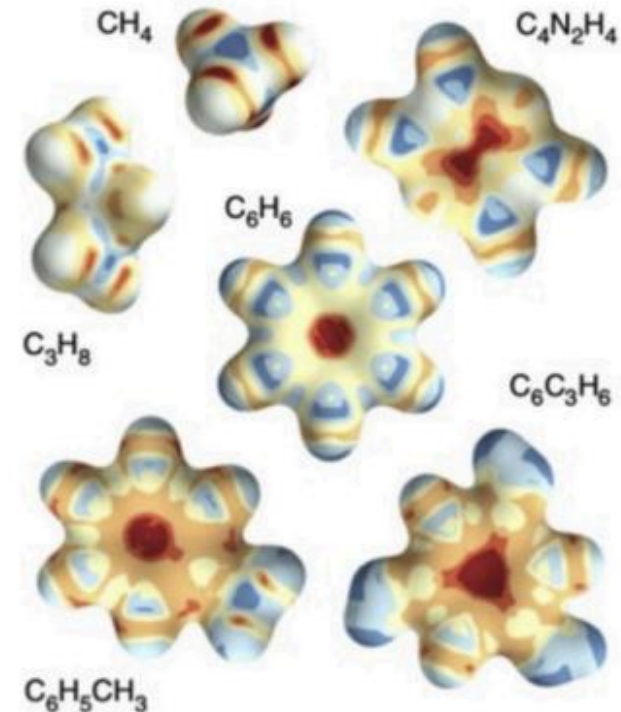
# Generate new scientific insights

Explanations patterns can help us learn about physical, biological, and chemical mechanisms

Which genes unexpectedly appear in disease diagnosis explanations?



Which chemical structures explain reaction properties?





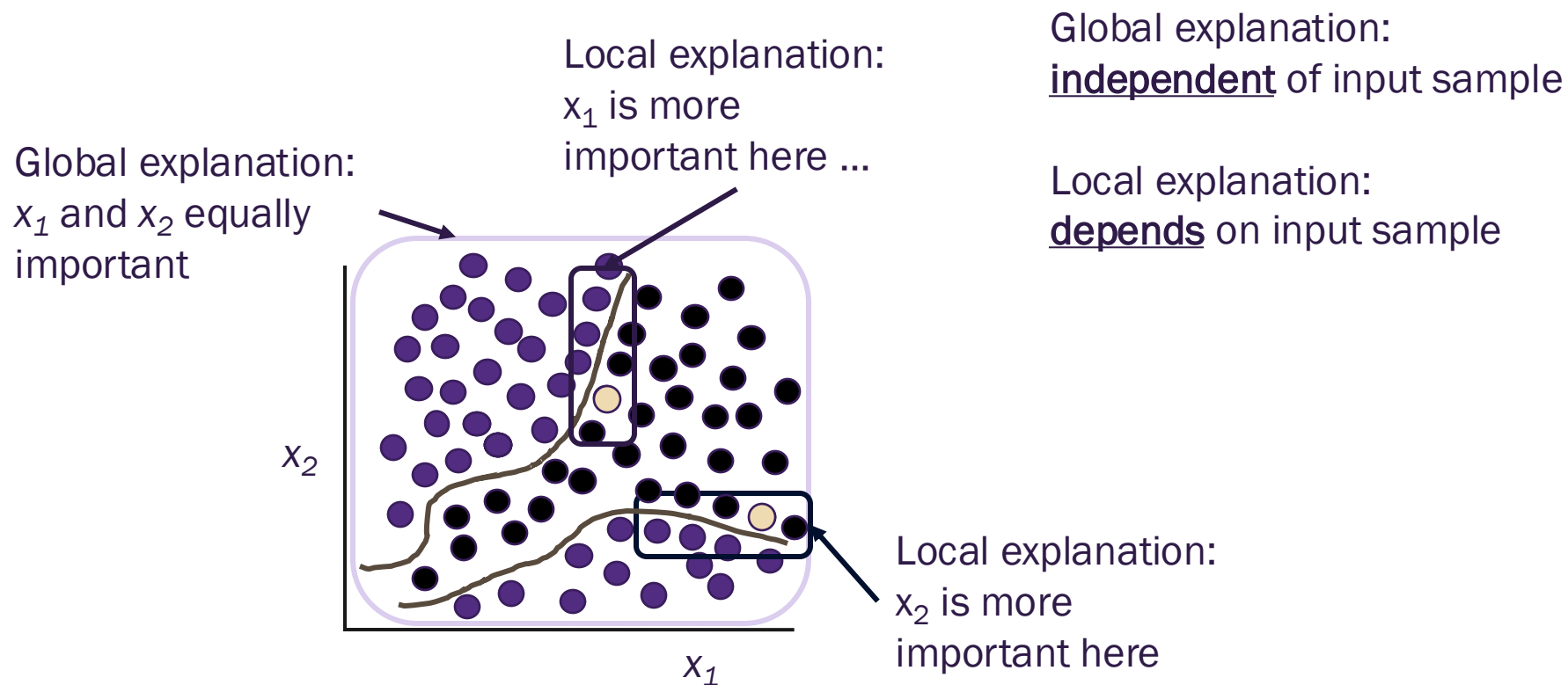
# XAI methods for tabular, image, text data



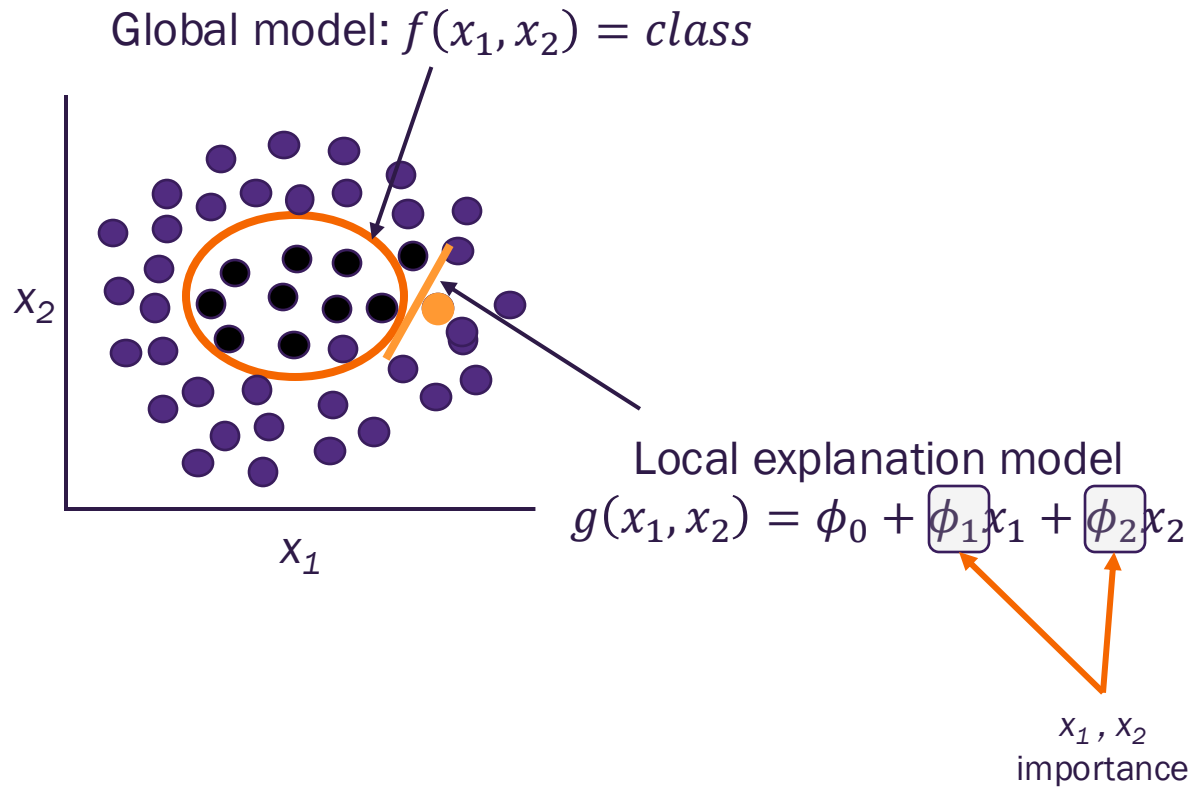


# Factual explanations

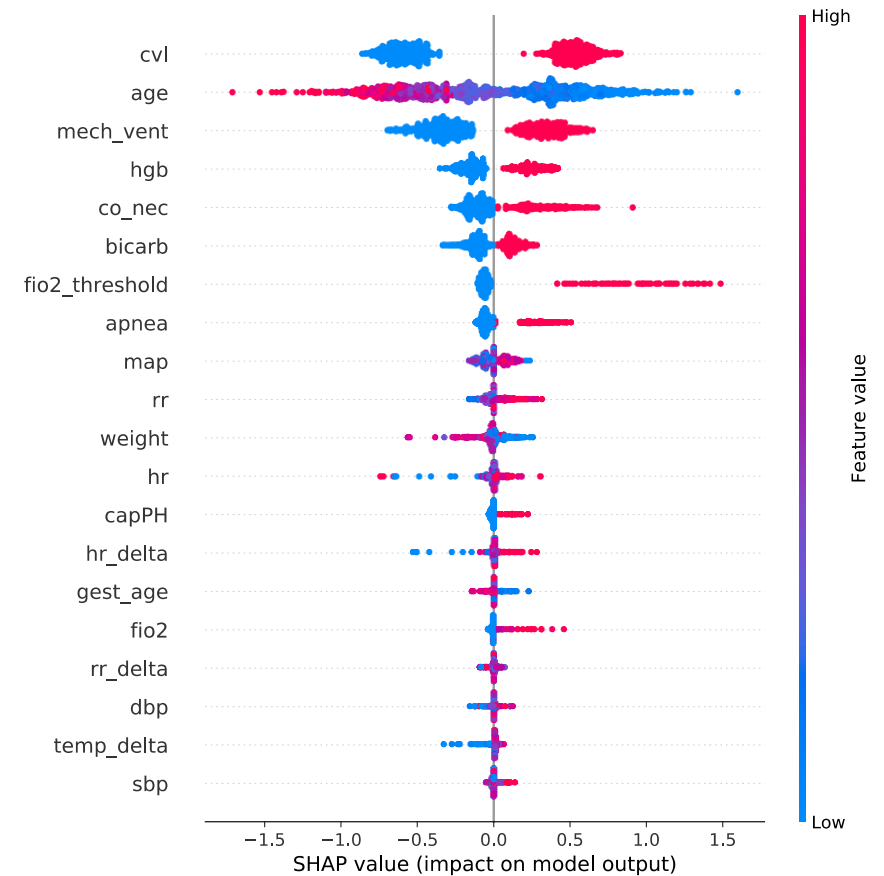
- Explanations in the form of features that most affect the model output
- Also referred to as *feature importance* methods
- Most common form of XAI



# Local feature importance via additive feature attribution



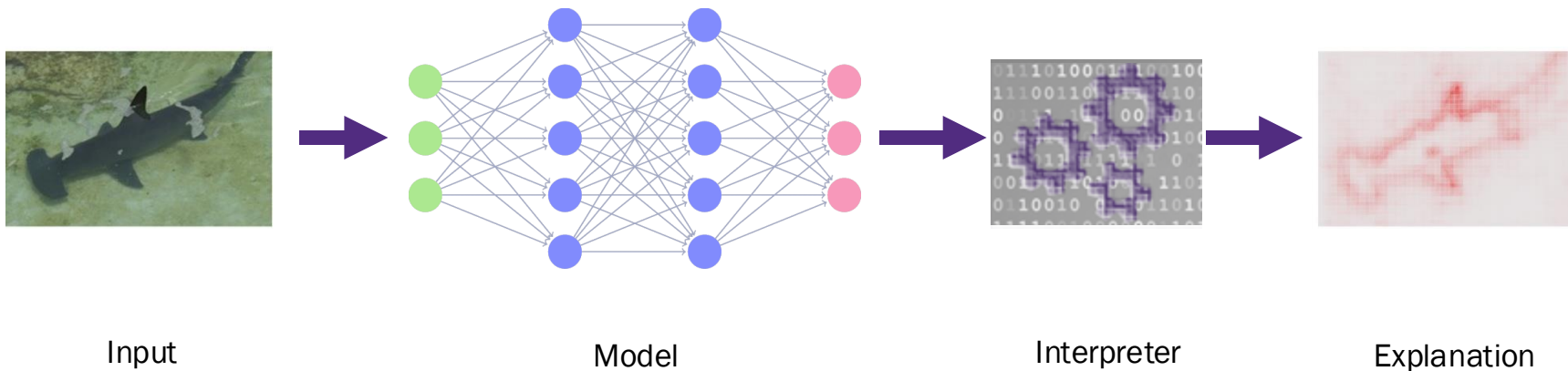
## Shapley Additive exPlanation (SHAP)



# Visualization local feature importance methods

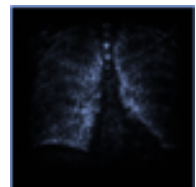
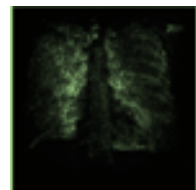
For a model  $f: R^d \rightarrow R^+$ , associate with each input element a relevance score,  $R_p(x)$ , that indicates importance

- Gradient methods:  $\sum_p \left( \partial f / \partial x_p \right)^2 = \|\nabla_x f(x)\|^2$
- Perturbation: Mask image areas
- Decomposition methods:  $f(x) = \sum_p R_p(x)$

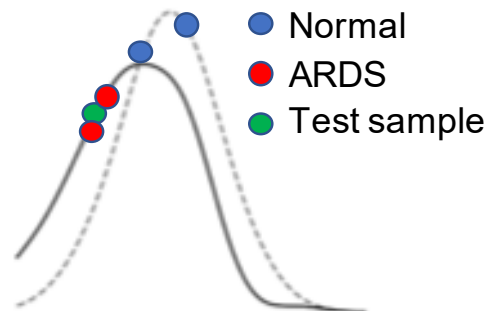


# Instance based explanations with influence functions

- Robust statistical methods that identify the training example(s) that most influence the current prediction
  - Deletion diagnostics (linear models)
  - Influence functions (twice differentiable loss function)
- Useful in model development as a means of identifying outliers with undue influence on predictions
- For deployed models can be used to indicate how important features of test sample compare to influential samples



SpO2/FiO2 Data Distribution



Report Findings Comparison

**Test Sample**

Opacification (Right)  
Pleural Apical Thickening

**Influential ARDS Training Samples**

Opacification

**Influential Normal Training Samples**

Opacification (negated)



# XAI methods for graphs





# Challenges for GNN explainability

- Graphs contain input node feature and structure (topology) information
- Existing XAI methods do not apply to discrete valued inputs (i.e., adjacency matrices)
- For some graphs, only the structure information is important (nodes may not have any features). How should structural be used for explanations?
- For node classification, how should neighbor nodes be used in explanations?
- Domain specific knowledge may be necessary for explanations.
  - Ex: Explaining predictions on molecular graphs may require different methods than on citation networks.

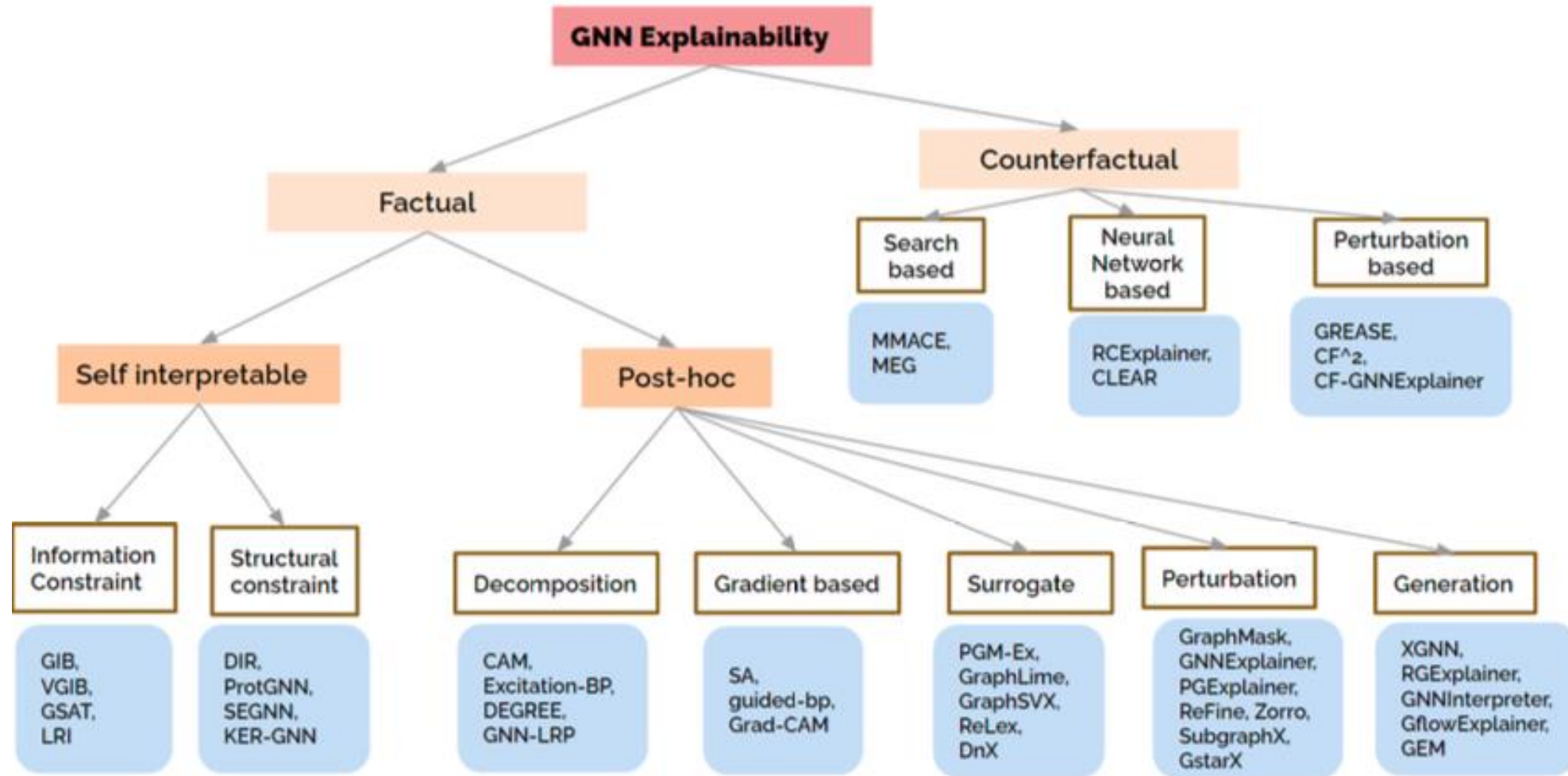


# Overview of GNN XAI methods

- Different XAI methods focus on different aspects of graph models
- Most answer a few questions:
  - Which input edges are most important?
  - Which input nodes are most important?
  - Which node features are most important?
  - What subgraphs are most important?

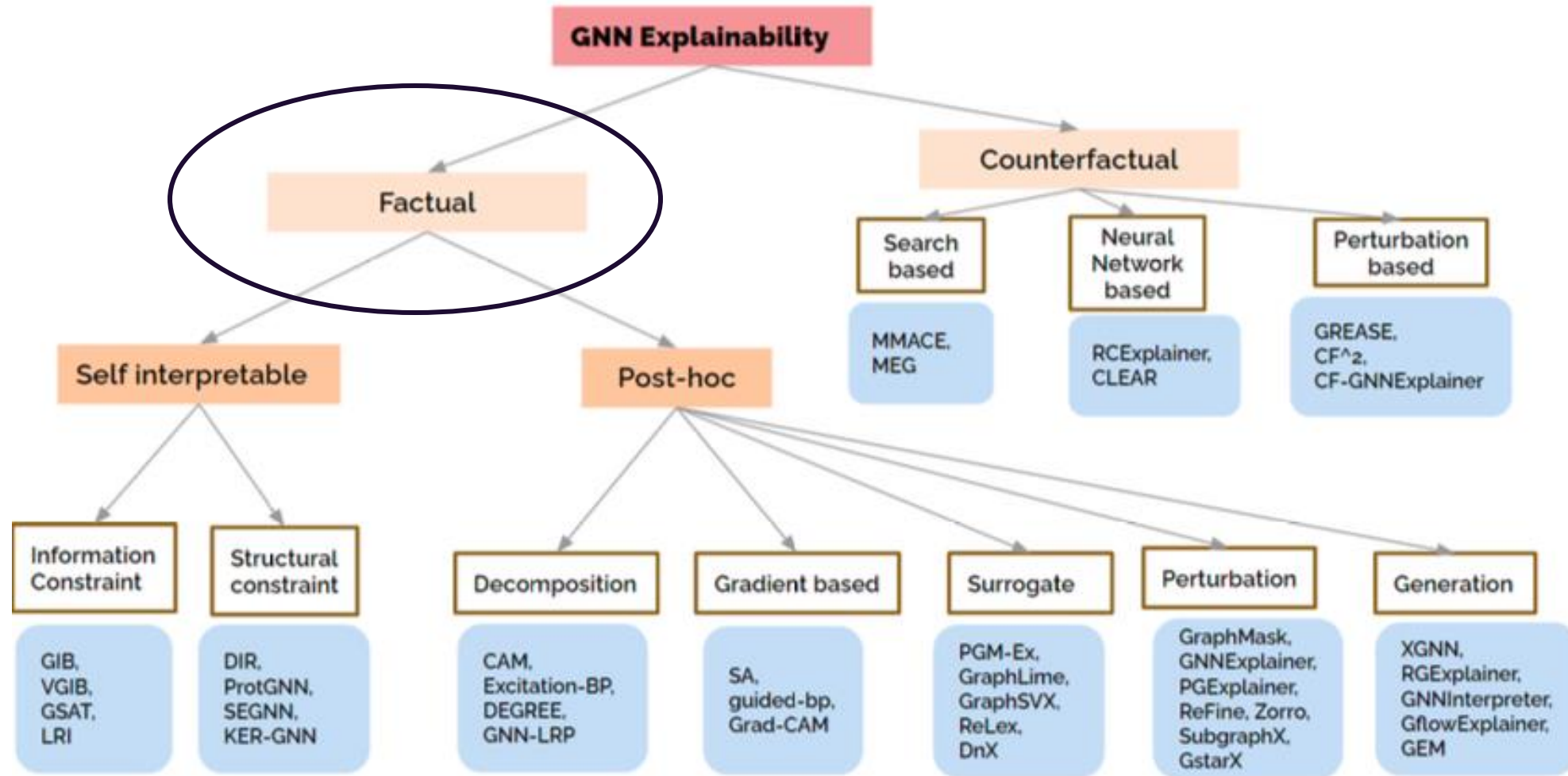


# GNN XAI method taxonomy





# GNN XAI method taxonomy





# Factual explanations on graphs

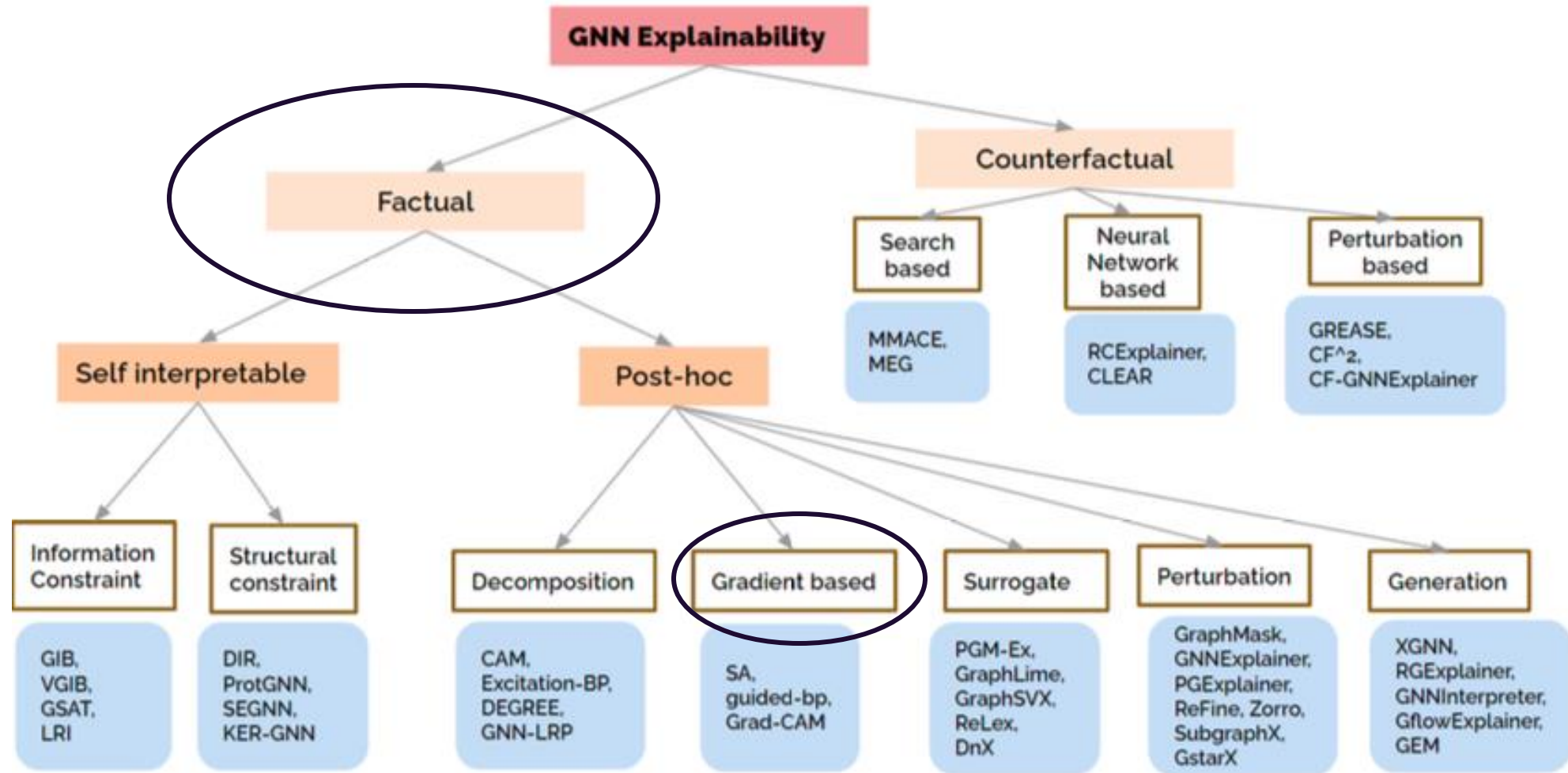
- Factual methods form an explanation as a **set of input features** (e.g., nodes) with the maximum influence on model output
- Post-hoc methods
  - Separate from the prediction model
  - Use the model input, output, and sometimes model weights
- Self-interpretable
  - Integrated into the model architecture
  - Usually have two components
    - Subgraph extraction module that finds the most informative subgraph of the input graph
    - Prediction module that uses the subgraph to make the prediction



# Evaluating factual methods

- Assume an explanation method indicates a set of important input features (e.g., nodes)
- **Fidelity**
  - Idea: removal of important features should reduce model performance
  - Example: Difference in model prediction accuracy averaged across all classes
- **Contrastivity**
  - Idea: Important features should differ across possible outcomes
  - Example: Hamming distance between binarized importance scores on nodes by class
- **Sparsity**
  - Ideas: explanations should be minimal
  - Example: the percentage of nodes in the graph identified as important

# GNN XAI method taxonomy





# Post-hoc methods

- Gradient methods
  - Motivation: gradient of the prediction with respect to the input represents how sensitive the prediction is to the input

How do we compute the gradients of the prediction for a GNN?



# Contrastive gradient saliency maps for GCNs

- Consider **graph classification task**
- Goal is to assess node importance
- Consider a GNN with
  - Arbitrary number of graph convolutional layers (GCNs)
  - Followed by a global mean pooling (GMP) layer
  - GMP layers input to a classifier (e.g., fully connected output layer)



# The math of graph convolution networks in matrix form

- Let  $N = |V|$  be the number of nodes in the graph  $G$  and  $\mathbf{H}^{(k-1)} = \begin{bmatrix} h_1^{k-1} \\ \vdots \\ h_N^{k-1} \end{bmatrix}$  be the matrix of latent node representations  $\forall v \in G$  at layer  $k - 1$

- We can write the update function over all nodes as:

$$\mathbf{H}^{(k)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}^{(k)})$$

where:

$$\tilde{\mathbf{A}} = \mathbf{D}^{-1/2}[\mathbf{A} + \mathbf{I}_N]\mathbf{D}^{1/2} \in \mathbb{R}^{N \times N}$$

$\mathbf{I}_N \in \mathbb{R}^{N \times N}$  is the identity matrix (adds self loops)

$\mathbf{D}$  is the diagonal degree matrix with  $\mathbf{D}_{ii} = \sum_j [\mathbf{A} + \mathbf{I}_N]_{ij}$

$$\mathbf{H}^{(k-1)} \in \mathbb{R}^{N \times d^{(k-1)}}$$

$$\mathbf{W}^{(k)} \in \mathbb{R}^{d^{(k-1)} \times d^{(k)}}$$

$$\mathbf{H}^{(k)} \in \mathbb{R}^{N \times d^{(k)}}$$

# Decomposing the GCN score

- Given the update function

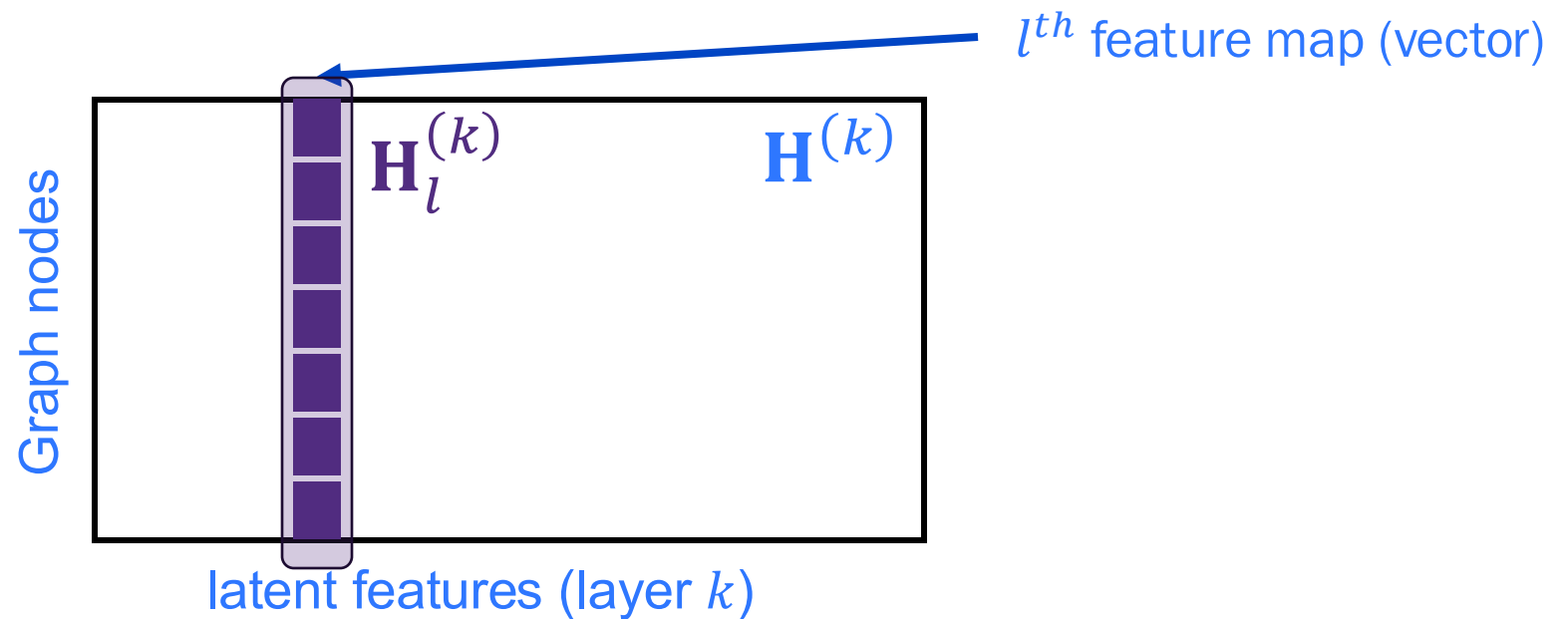
$$\mathbf{H}^{(k)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}^{(k)}) \in \mathbb{R}^{N \times d^{(k)}}$$

we can write the  $l^{th}$  feature map,  $\mathbf{H}_l^{(k)}$ , at layer  $k$ , as

$$\mathbf{H}_l^{(k)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}_l^{(k)}) \in \mathbb{R}^{N \times 1}$$

where  $\mathbf{W}_l^{(k)} \in \mathbb{R}^{d^{(k-1)} \times 1}$  is the  $l^{th}$  column of  $\mathbf{W}^{(k)}$

Recall, you can view the node embedding matrix,  $\mathbf{H}^{(k)}$ , as a table where the rows are nodes and the columns are features



# Decomposing the GCN score

- Given the update function

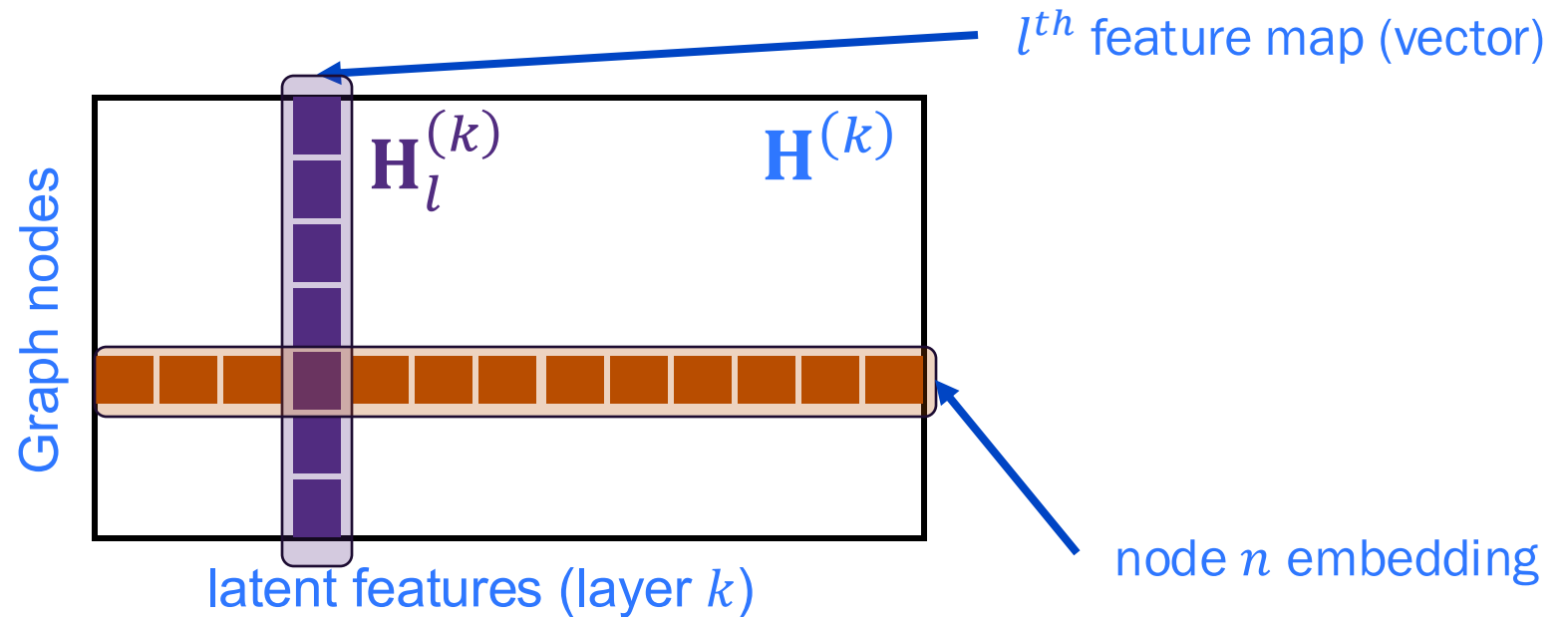
$$\mathbf{H}^{(k)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}^{(k)}) \in \mathbb{R}^{N \times d^{(k)}}$$

we can write the  $l^{th}$  feature map,  $\mathbf{H}_l^{(k)}$ , at layer  $k$ , as

$$\mathbf{H}_l^{(k)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}_l^{(k)}) \in \mathbb{R}^{N \times 1}$$

where  $\mathbf{W}_l^{(k)} \in \mathbb{R}^{d^{(k-1)} \times 1}$  is the  $l^{th}$  column of  $\mathbf{W}^{(k)}$

Recall, you can view the node embedding matrix,  $\mathbf{H}^{(k)}$ , as a table where the rows are nodes and the columns are features



# Decomposing the GCN score

- Given the update function

$$\mathbf{H}^{(k)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}^{(k)}) \in \mathbb{R}^{N \times d^{(k)}}$$

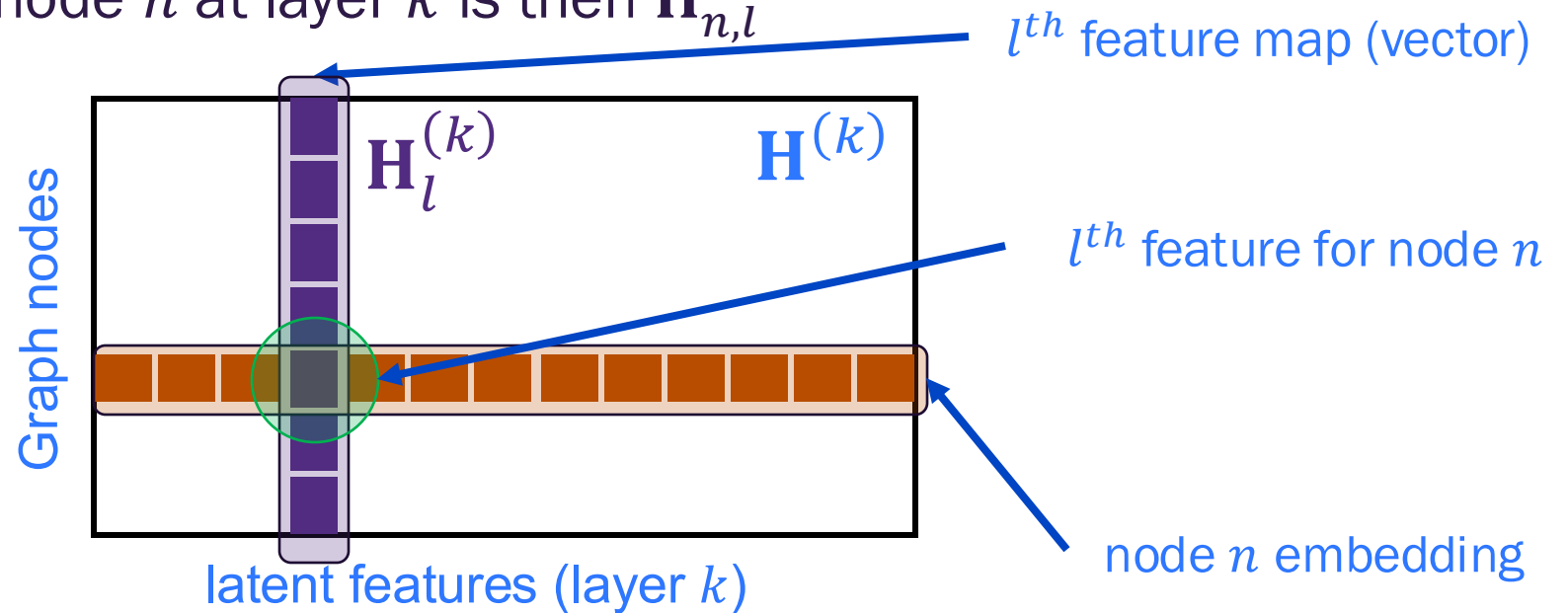
we can write the  $l^{th}$  feature map,  $\mathbf{H}_l^{(k)}$ , at layer  $k$ , as

$$\mathbf{H}_l^{(k)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}_l^{(k)}) \in \mathbb{R}^{N \times 1}$$

where  $\mathbf{W}_l^{(k)} \in \mathbb{R}^{d^{(k-1)} \times 1}$  is the  $l^{th}$  column of  $\mathbf{W}^{(k)}$

- The  $l^{th}$  feature for node  $n$  at layer  $k$  is then  $\mathbf{H}_{n,l}^{(k)}$

Recall, you can view the node embedding matrix,  $\mathbf{H}^{(k)}$ , as a table where the rows are nodes and the columns are features





# Contrastive gradient saliency maps for GCNs

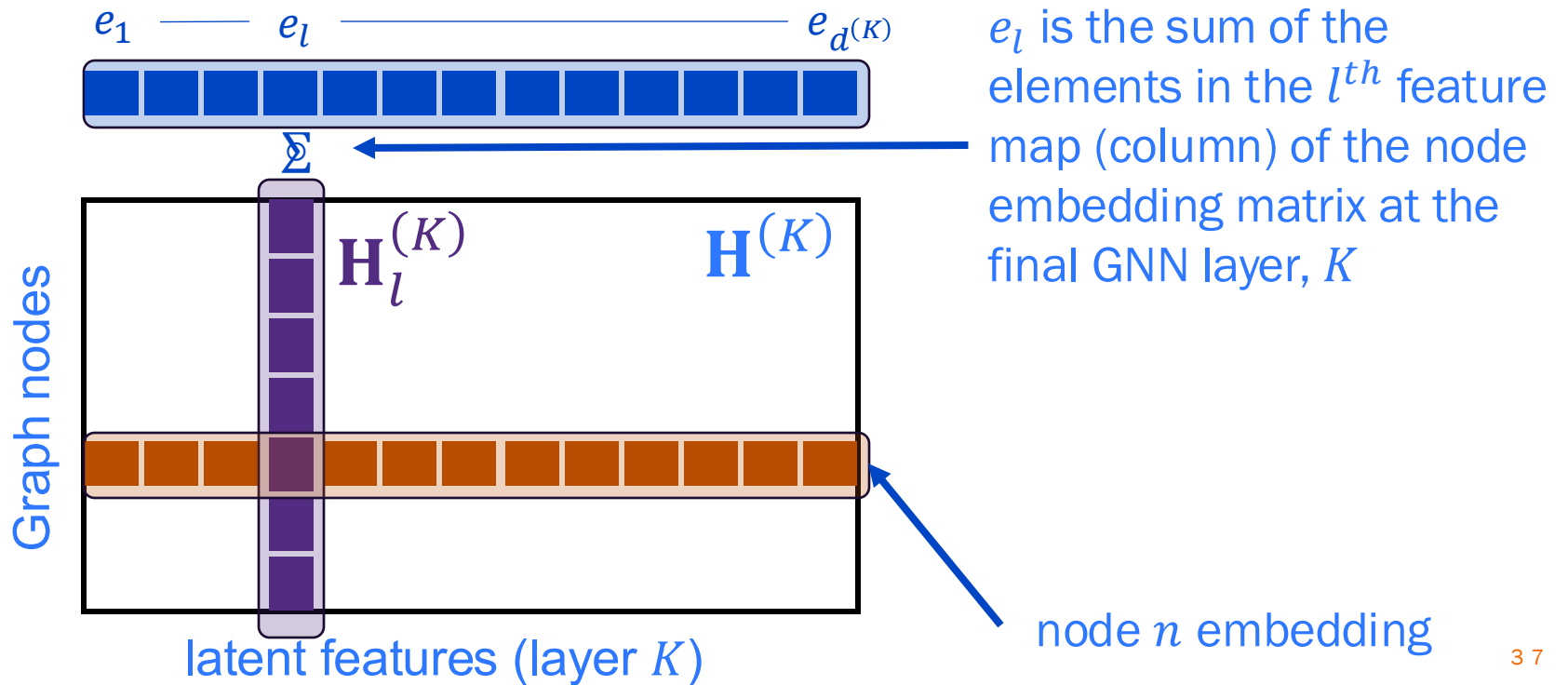
- Consider **graph classification task**
- Goal is to assess node importance
- Consider a GNN with
  - Arbitrary number of graph convolutional layers (GCNs)
  - **Followed by a global mean pooling (GMP) layer**
  - GMP layers input to a classifier (e.g., fully connected output layer)

# Decomposing the GCN score

- Let  $\mathbf{H}^{(K)}$  be the **final GCN layer**
- The  $l^{th}$  **global mean pooling feature**,  $e_l$ , is given by

$$e_l = \frac{1}{N} \sum_{n=1}^N \mathbf{H}_{n,l}^{(K)}$$

where  $N$  is the number of nodes in the graph





# Contrastive gradient saliency maps for GCNs

- Consider **graph classification task**
- Goal is to assess node importance
- Consider a GNN with
  - Arbitrary number of graph convolutional layers (GCNs)
  - Followed by a global mean pooling (GMP) layer
  - **GMP layers input to a classifier (e.g., fully connected output layer)**

# Decomposing the GCN score

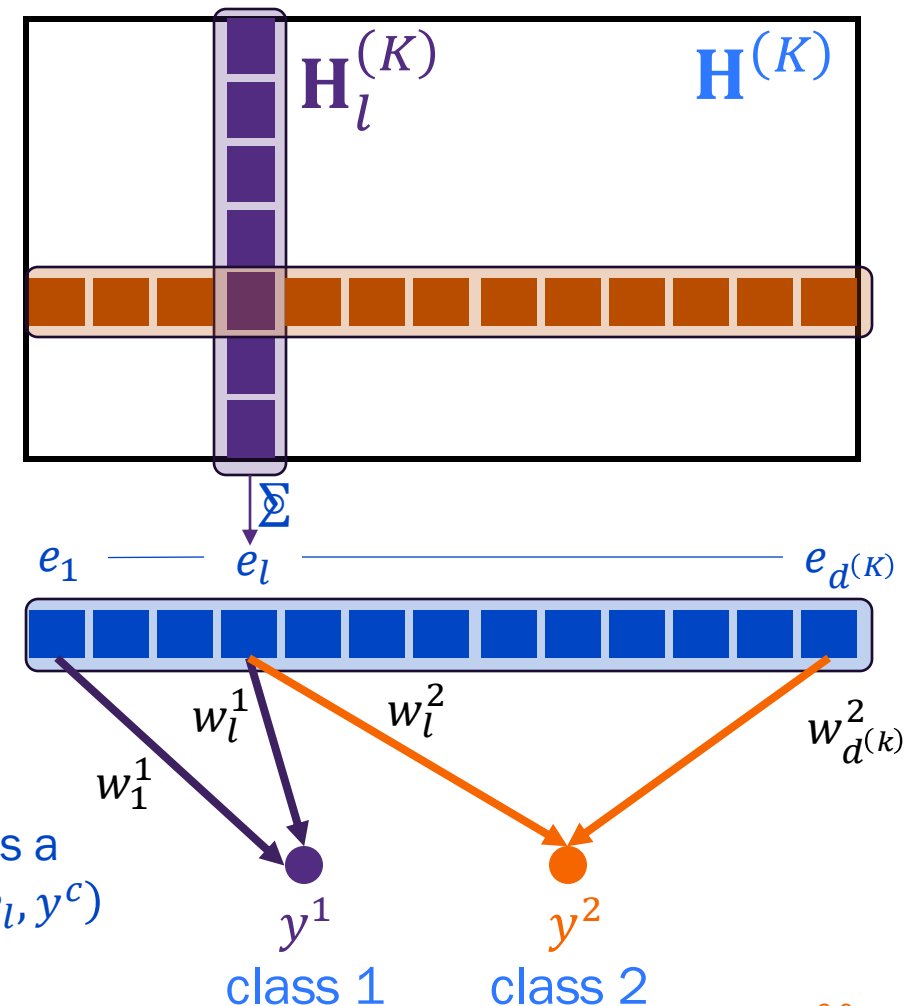
- Let  $\mathbf{H}^{(K)}$  be the final GCN layer
- The  $l^{th}$  global mean pooling feature,  $e_l$ , is given by

$$e_l = \frac{1}{N} \sum_{n=1}^N \mathbf{H}_{n,l}^{(K)}$$

where  $N$  is the number of nodes in the graph

- The class scores (logits) are now given by

$$y^c = \sum_{l=1}^{d^{(K)}} w_l^c e_l$$



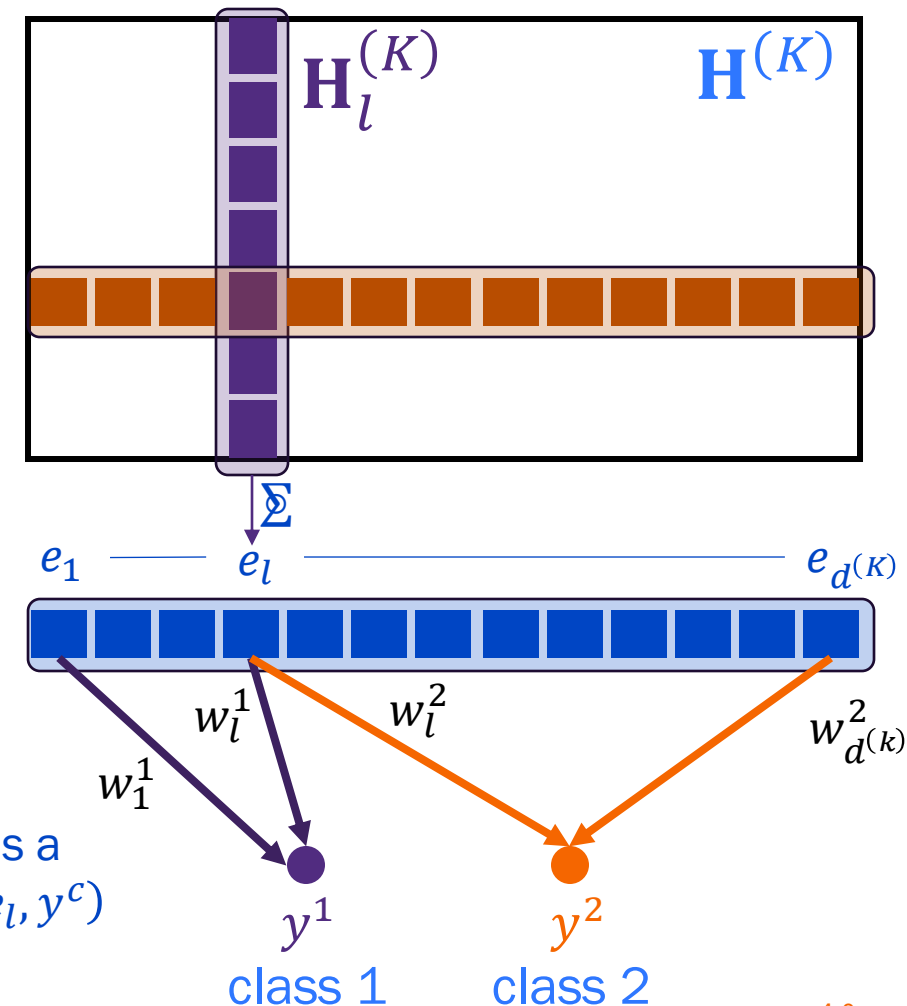
# Contrastive gradient saliency maps for GCNs

- The class scores (logits) are now given by

$$y^c = \sum_{l=1}^{d^{(K)}} w_l^c e_l$$

- Finally, the importance score for node  $n$  for class  $c$  is given by:

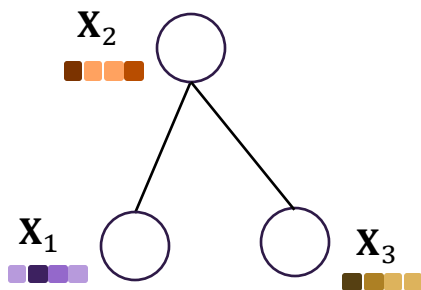
$$L_{CGS}^c(n) = \left\| \text{ReLU} \left( \frac{\delta y^c}{\delta \mathbf{X}_n} \right) \right\|_l$$



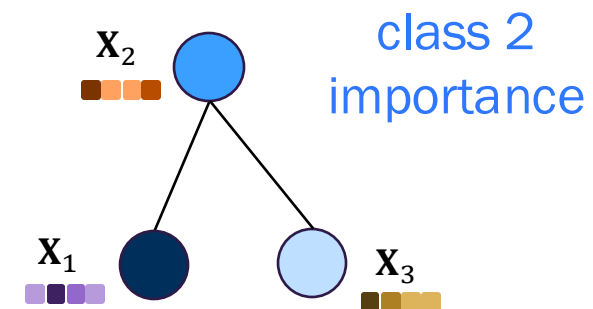
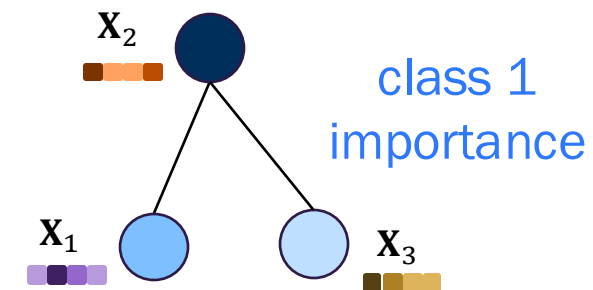
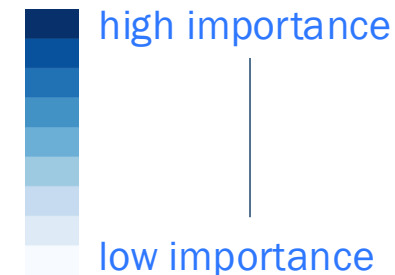


# Contrastive gradient saliency maps for GCNs

- The node importance scores can be visualized as a node heat map



$$L_{CGS}^c(n) = \left\| \text{ReLU} \left( \frac{\delta y^c}{\delta \mathbf{X}_n} \right) \right\|_l$$



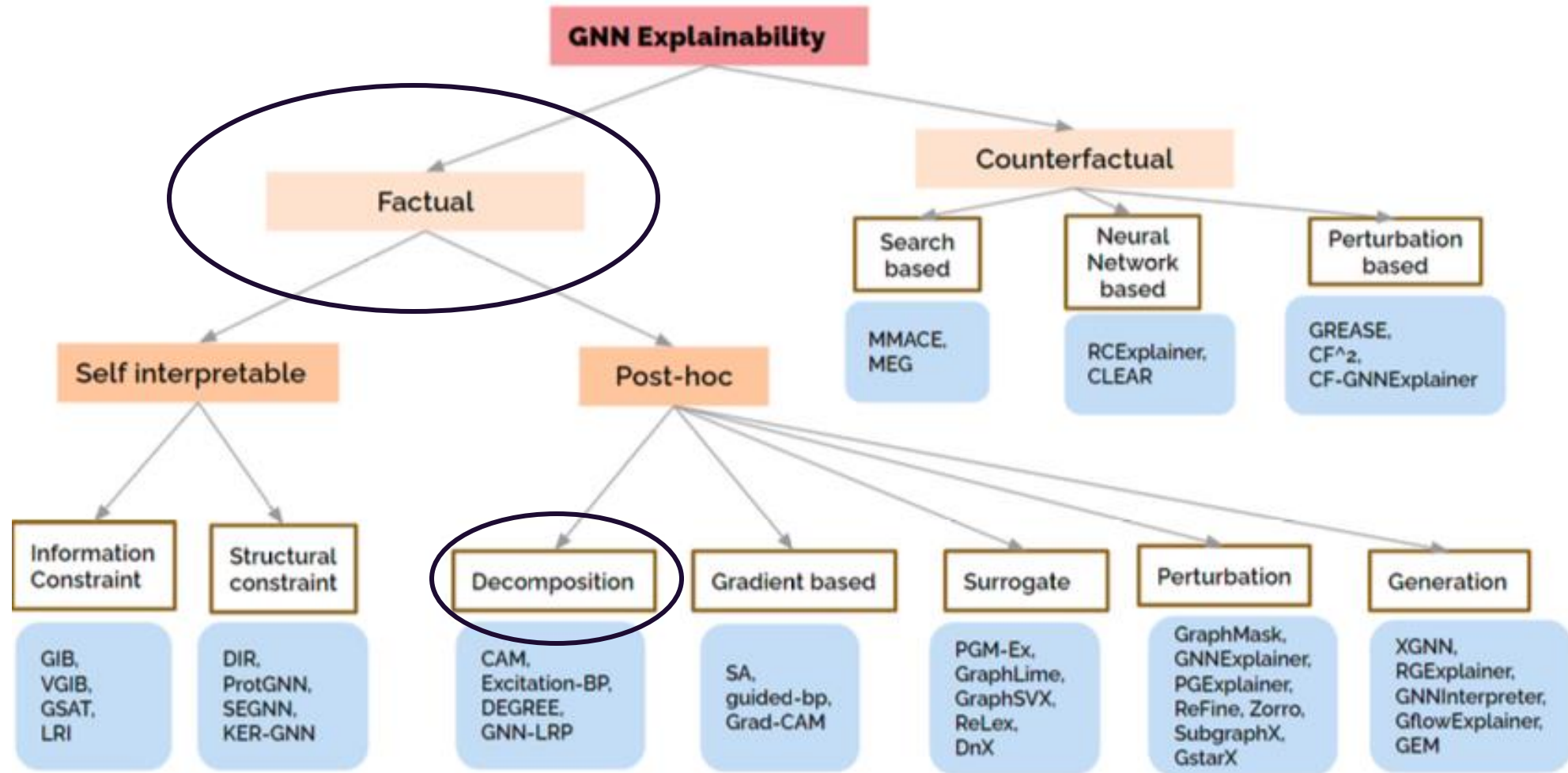


# Summary of contrastive gradient saliency maps for GCNs

- The node importance score is taken to be the gradient (partial derivative) of the class probability prediction with respect to the node input features
- **Many limitations**
  - Low fidelity: Very sensitive to small perturbations that don't change class prediction
  - Low sparsity: Often indicates high importance for many nodes in the graph
  - Low contrastivity: Often indicates the same nodes are important for every class



# GNN XAI method taxonomy





# Post-hoc methods

- Decomposition methods
  - Treat model predictions as a score that can be decomposed and distributed across model component all the way back to the input
  - The portion of the score allocated to a component is its importance
  - Methods vary by the way in which the score is decomposed
  - White-box: all such methods require access to the model weights

How should we decompose the GNN output score?

# Decomposing the GCN score

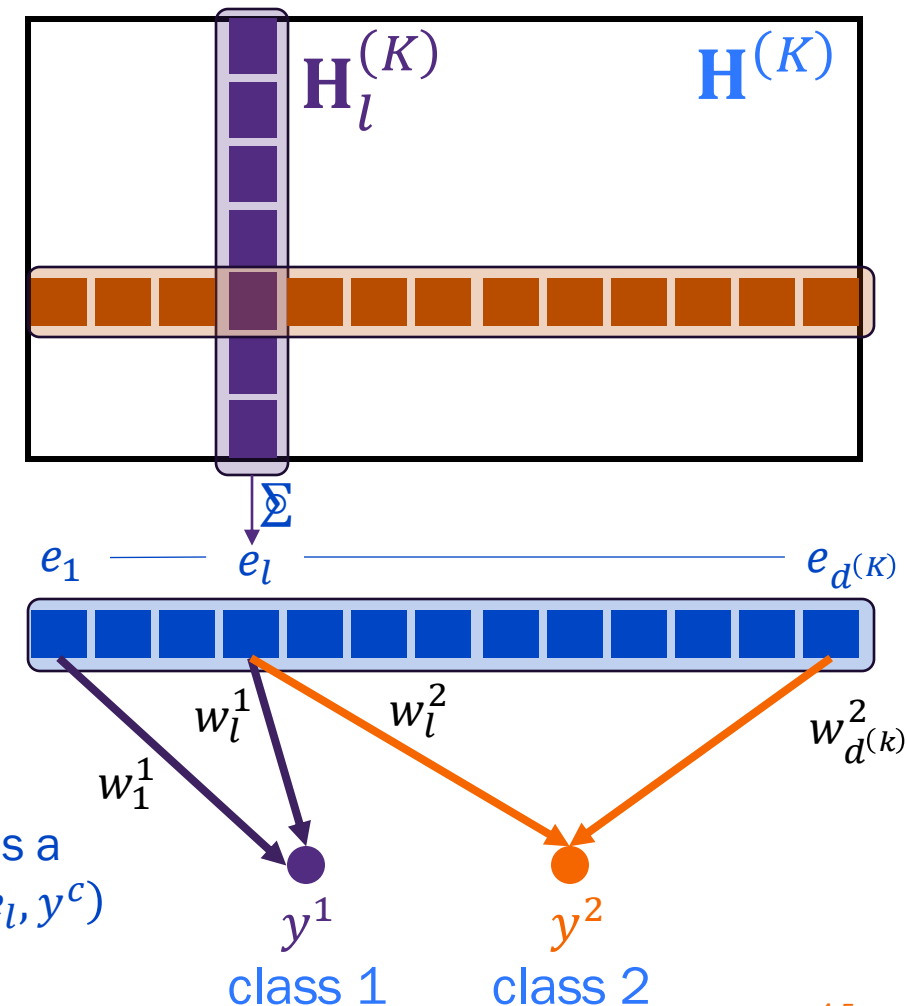
- Let  $\mathbf{H}^{(K)}$  be the **final GCN layer**
- The  $l^{th}$  **global mean pooling feature**,  $e_l$ , is given by

$$e_l = \frac{1}{N} \sum_{n=1}^N \mathbf{H}_{n,l}^{(K)}$$

where  $N$  is the number of nodes in the graph

- The class scores (logits) are now given by

$$y^c = \sum_{l=1}^{d^{(K)}} w_l^c e_l$$



(not shown) there is a weight,  $w_l^c$  for pair  $(e_l, y^c)$

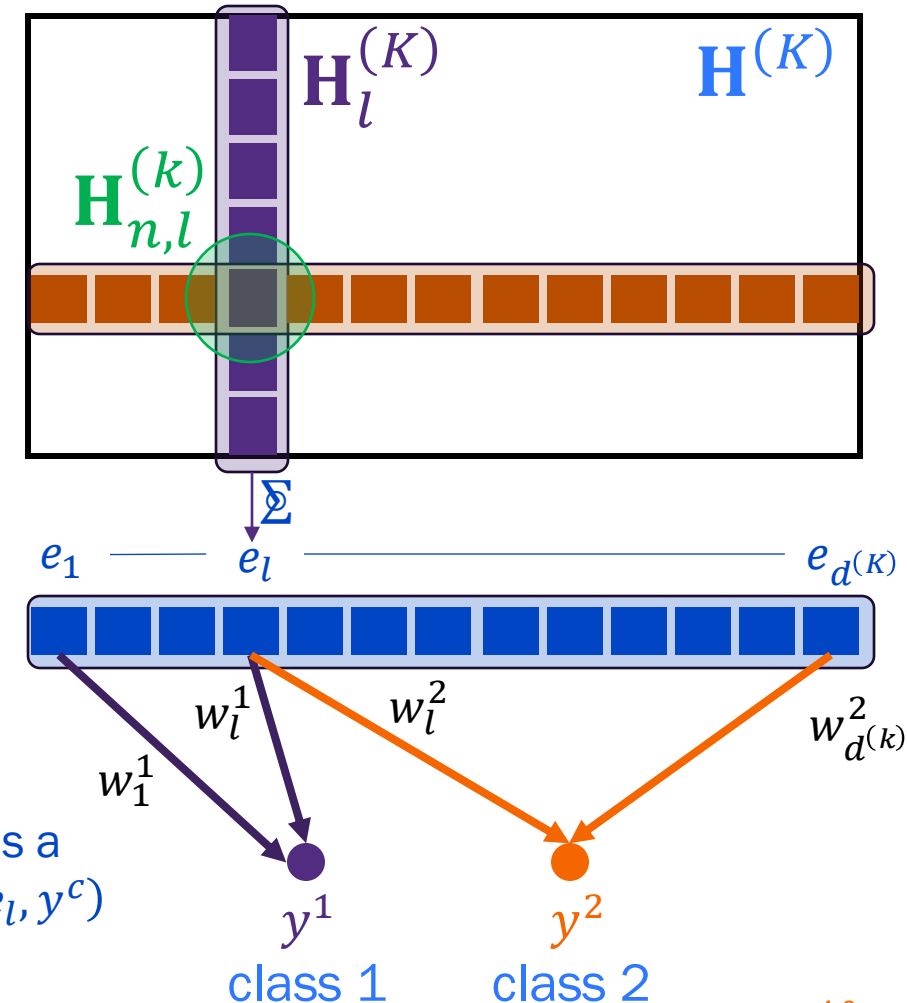
# Class activation maps (CAM)

- Consider the  $l^{th}$  feature map at the final GCN layer,  $K$

$$\mathbf{H}_l^{(K)} = \sigma \left( \tilde{\mathbf{A}} \mathbf{H}^{(k-1)} \mathbf{W}_l^{(K)} \right) \in \mathbb{R}^{N \times 1}$$

- The CAM importance score for node  $n$  for class  $c$  is given by the **final node embedding** scaled by the **output layer weights**:

$$L_{CAM}^c(n) = \text{ReLU} \left( \sum_{l=1}^{d^{(K)}} w_l^c \mathbf{H}_{n,l}^{(K)} \right)$$

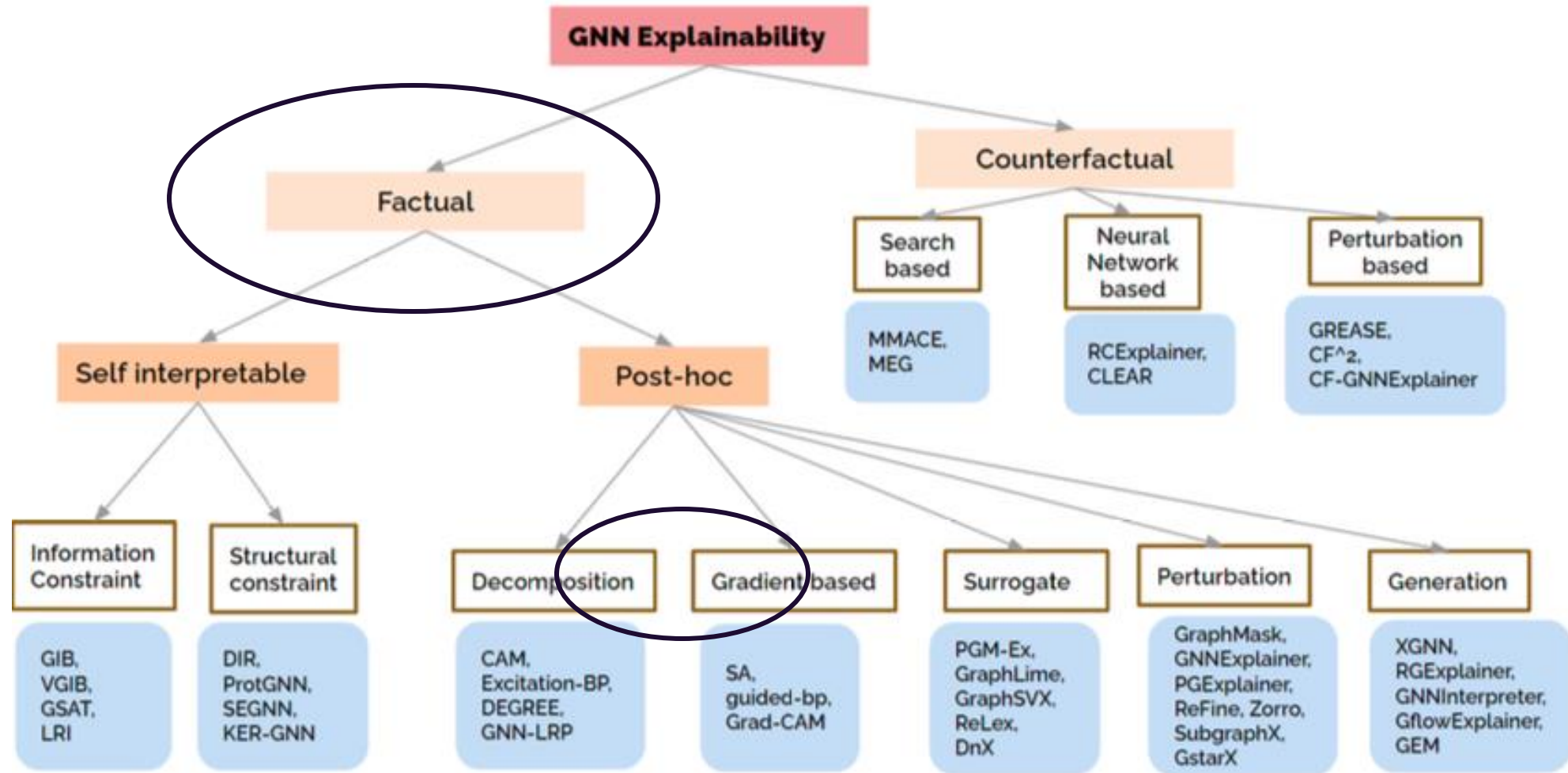




# Summary of class activation maps (CAM) for GCN

- CAM scoring represents the contribution of the final node “feature map” (embedding) to the class score
- Typically displayed as a node heat map
- Strengths
  - High contrastivity: often results in different node importance across different classes
  - High fidelity: often results in identifying nodes, that if removed, reduce model accuracy
- Limitations
  - Low sparsity: often identifies many nodes as important which may be problematic on large graphs

# GNN XAI method taxonomy





# What about layers before the final layer?

- Both the contrastive gradient and class activation map methods consider importance based on the final node embedding layer
- This can inform
  - Hierarchical feature learning
    - Do lower layers focus on local structure?
    - Do higher layers capture larger node groups?
  - Model debugging
    - Many nodes important at early layers
    - Node importance doesn't change across layers

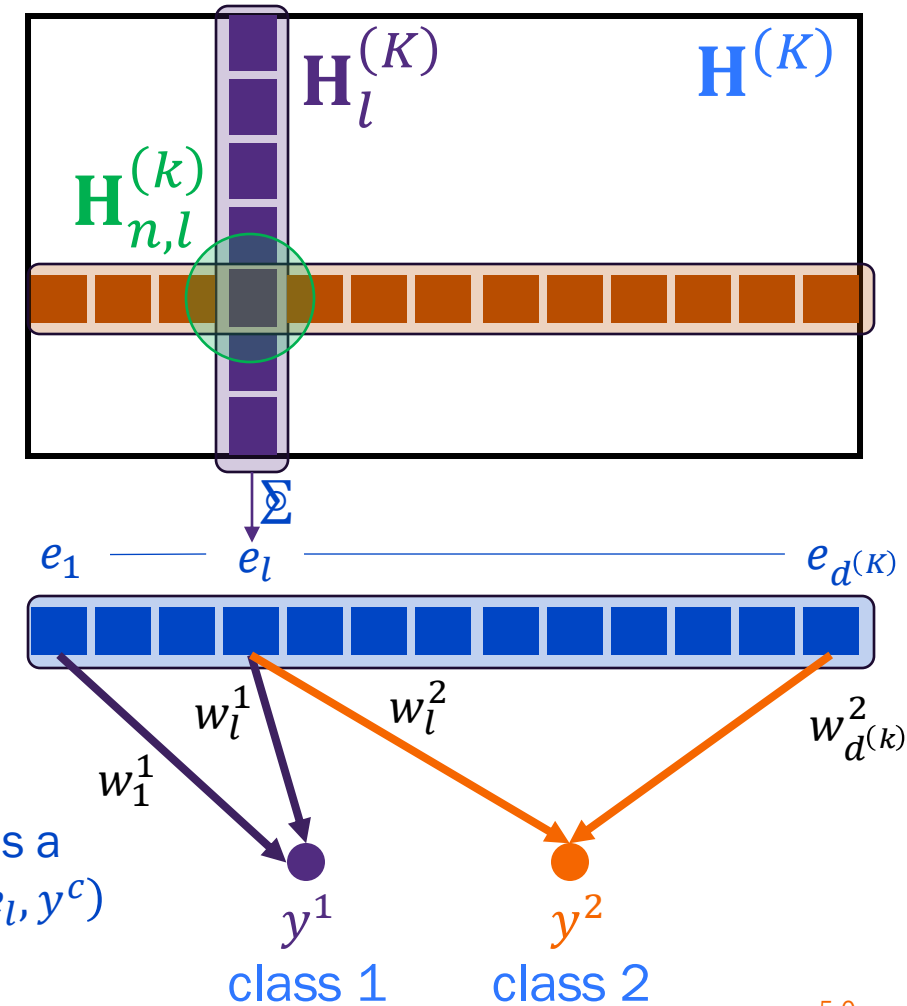
# Grad-CAM

- Consider the  $l^{th}$  feature map at the final GCN layer,  $K$

$$\mathbf{H}_l^{(K)} = \sigma \left( \tilde{\mathbf{A}} \mathbf{H}^{(k-1)} \mathbf{W}_l^{(K)} \right) \in \mathbb{R}^{N \times 1}$$

- The CAM importance score for node  $n$  for class  $c$  is given by the **final node embedding** scaled by the **output layer weights**:

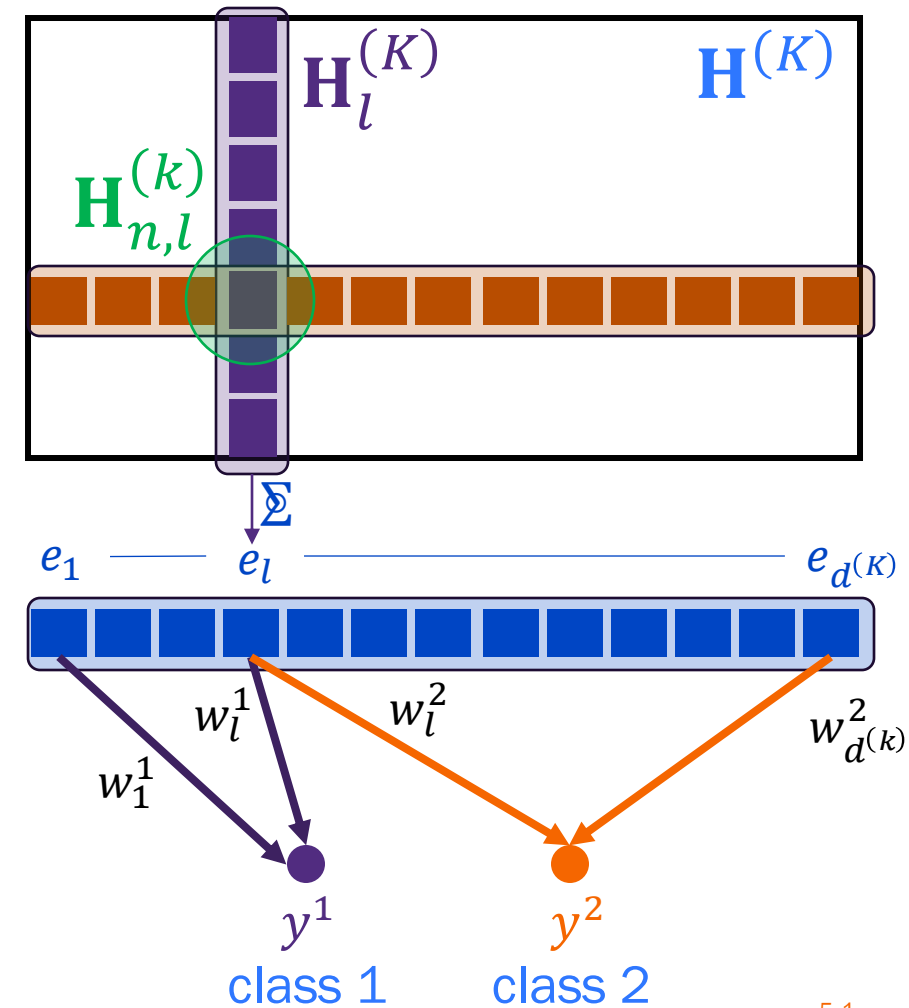
$$L_{CAM}^c(n) = \text{ReLU} \left( \sum_{l=1}^{d^{(K)}} w_l^c \mathbf{H}_{n,l}^{(K)} \right)$$



# Grad-CAM

- Recall, the GNN score was decomposed as

$$y^c = \sum_{l=1}^{d^{(K)}} w_l^c e_l = \frac{1}{N} \sum_{l=1}^{d^{(K)}} w_l^c \sum_{n=1}^N \mathbf{H}_{n,l}^{(K)}$$



# Grad-CAM

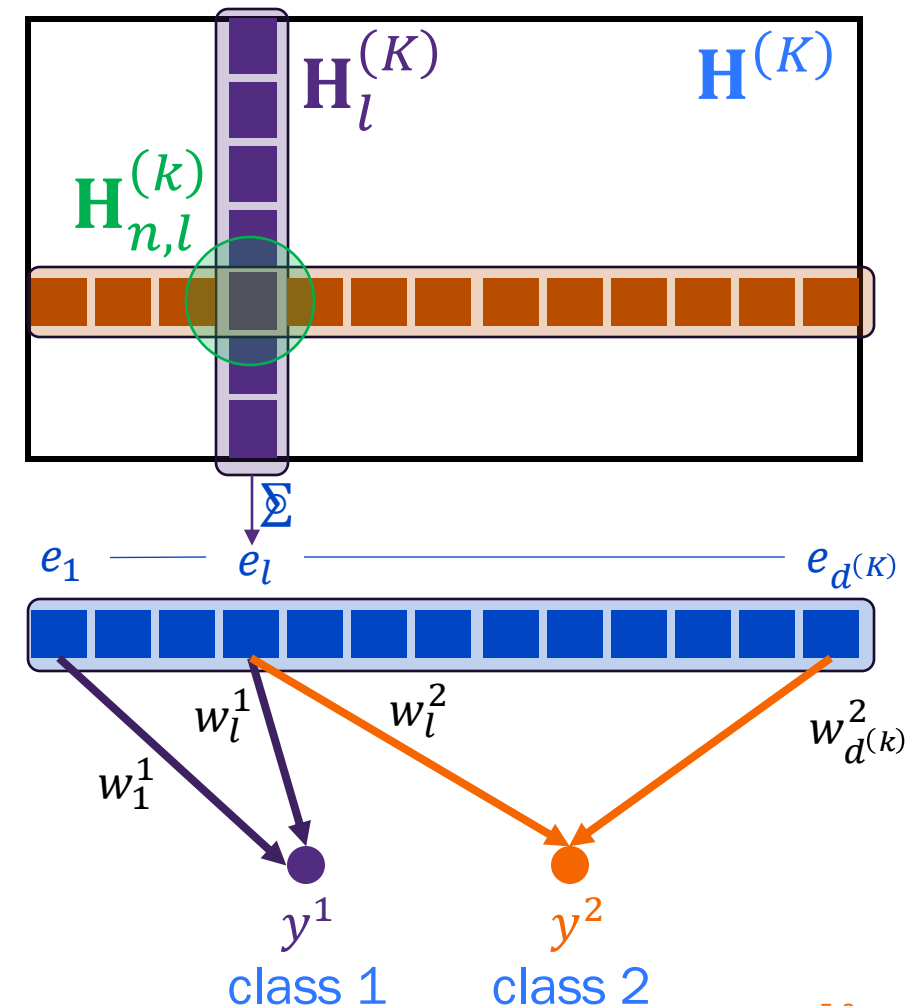
- Recall, the GNN score was decomposed as

$$y^c = \sum_{l=1}^{d^{(K)}} w_l^c e_l = \frac{1}{N} \sum_{l=1}^{d^{(K)}} w_l^c \sum_{n=1}^N \mathbf{H}_{n,l}^{(K)}$$

- Define class,  $c$ , and layer,  $k$ , specific weights for the  $l^{th}$  feature (node embedding space)

$$\alpha_l^{k,c} = \frac{1}{N} \sum_{n=1}^N \frac{\delta y^c}{\delta \mathbf{H}_{n,l}^{(k)}}$$

sensitivity of class probability logit to  
changes in the node's  $l^{th}$  feature  
value at layer  $k$



# Grad-CAM

- Recall, the GNN score was decomposed as

$$y^c = \sum_{l=1}^{d^{(K)}} w_l^c e_l = \frac{1}{N} \sum_{l=1}^{d^{(K)}} w_l^c \sum_{n=1}^N \mathbf{H}_{n,l}^{(K)}$$

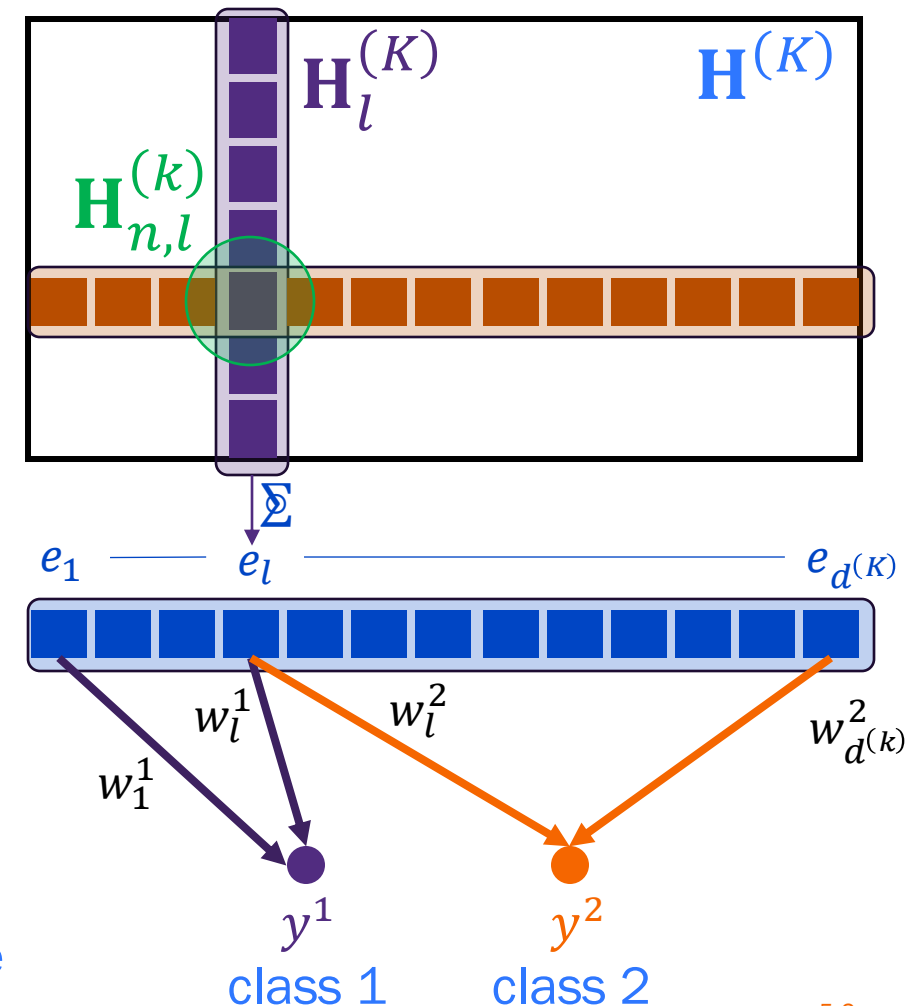
- Define class,  $c$ , and layer,  $k$ , specific weights for the  $l^{th}$  feature (node embedding space)

$$\alpha_l^{k,c} = \frac{1}{N} \sum_{n=1}^N \frac{\delta y^c}{\delta \mathbf{H}_{n,l}^{(k)}}$$

- The importance score for node  $n$  at layer  $k$  is

$$L_{GradCAM}^c(k, n) = ReLU \left( \sum_{l=1}^{d^{(k)}} \alpha_l^{k,c} \mathbf{H}_{n,l}^{(k)} \right)$$

sum of the value of node's  $l^{th}$  feature value at layer  $k$   
weighted by sensitivity of output to changes in the feature



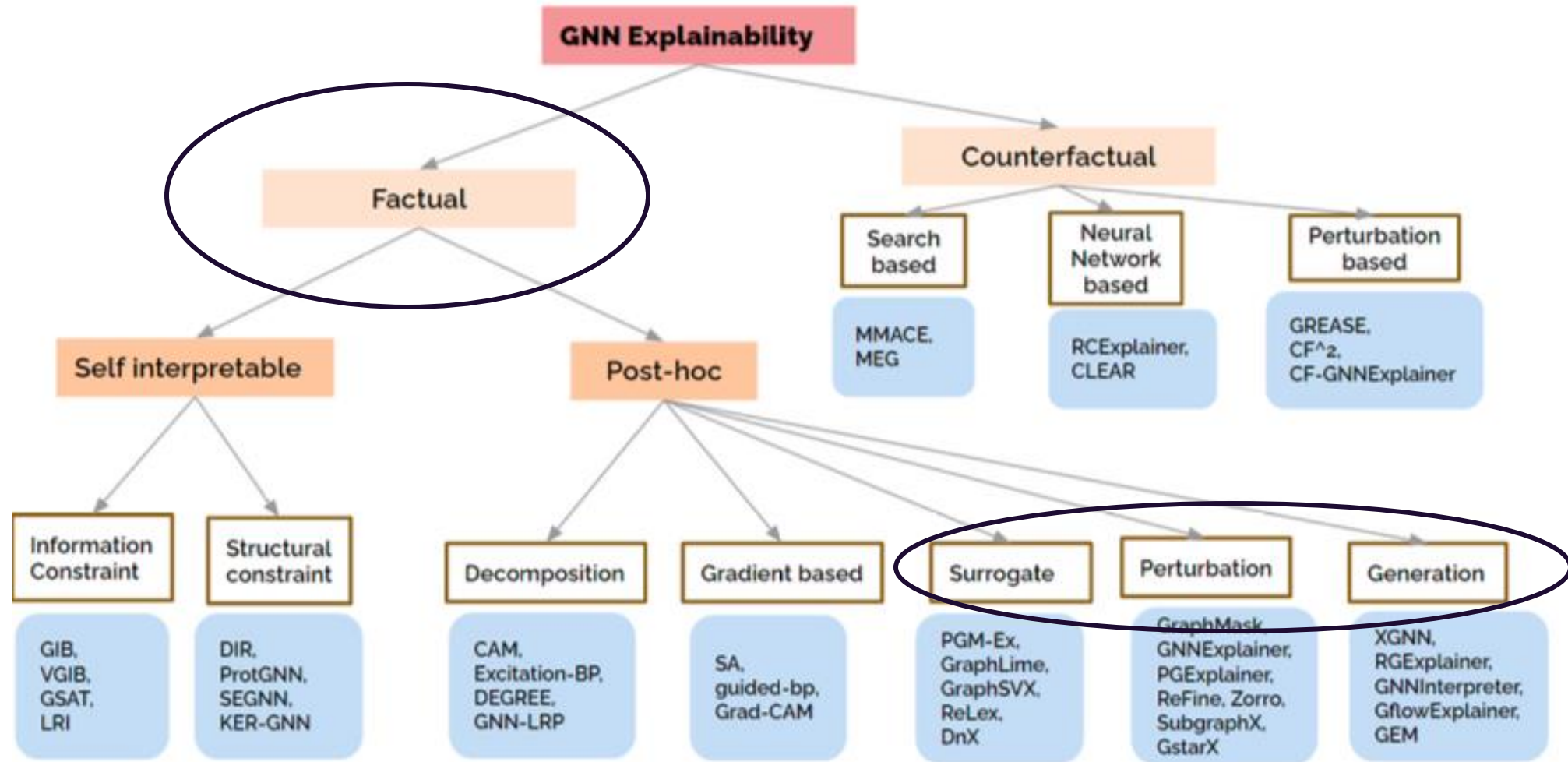


# Summary of Grad-CAM for GCN

- Grad-CAM scoring represents the contribution of a nodes “feature map” (embedding) at a given layer to the class score
- Grad-CAM at the final layer is equivalent to CAM
- Typically display heat map for final layer, with analysis at lower layer for detailed interpretations / model debugging
- Strengths
  - Same as CAM for final layer (high contrastivity, fidelity)
  - Enables layer wise analysis for debugging, finer grain interpretation of feature importance
- Limitations
  - Low sparsity at final layer (same as CAM)



# GNN XAI method taxonomy





# Surrogate methods for explaining GNNs

- Fit an interpretable surrogate model in the locality of the prediction
  - Given a graph  $G$  and a prediction on input  $x$  (i.e., a node or edge) generate predictions near  $x$  and record model predictions
  - Train the surrogate model and interpret important features

| Method              | Local Dataset extraction               | Surrogate model                      |
|---------------------|----------------------------------------|--------------------------------------|
| GraphLime [34]      | N-hop neighbor nodes                   | HSIC Lasso                           |
| PGMExplainer [97]   | Random node feature perturbation       | Bayesian network                     |
| ReLex [124]         | Random sampling of connected subgraphs | GCN                                  |
| DistilnExplain [75] | Entire dataset                         | Knowledge distilled Simple GCN [105] |
| GraphSVX [17]       | Input perturbations via mask generator | Weighted Linear Regression (WLR)     |



# Perturbation methods for explaining GNNs

- Identify important subgraphs (i.e. node and edge subsets) by perturbing the input graph
- Two general steps
  - For an input graph  $G$ , extraction module identifies a subgraph  $G_s$
  - Model prediction  $y_s$  on the subgraph compared to score  $y$
  - Training teaches extraction model to find subgraphs  $G_s$  that
    - minimize difference between  $y_s$  and  $y$
    - maintain sparsity of  $G_s$

| Method            | Subgraph Extraction Strategy            | Scoring function | Constraints              |
|-------------------|-----------------------------------------|------------------|--------------------------|
| GNExplainer [115] | Continuous relaxation                   | MI               | Size                     |
| SubgraphX [122]   | Monte Carlo Tree Search                 | SV               | Size, connectivity       |
| GraphMask [82]    | Layer-wise parameterized edge selection | $L_0$ norm       | Prediction divergence    |
| PGExplainer [56]  | Parameterized edge selection            | MI               | Size and/or connectivity |
| Zorro [23]        | Greedy selection                        | Fidelity         | Threshold fidelity       |
| ReFine [100]      | Parameterized edge attribution          | MI               | Number of edges          |
| GstarX [123]      | Monte Carlo sampling                    | HN-value         | Size                     |



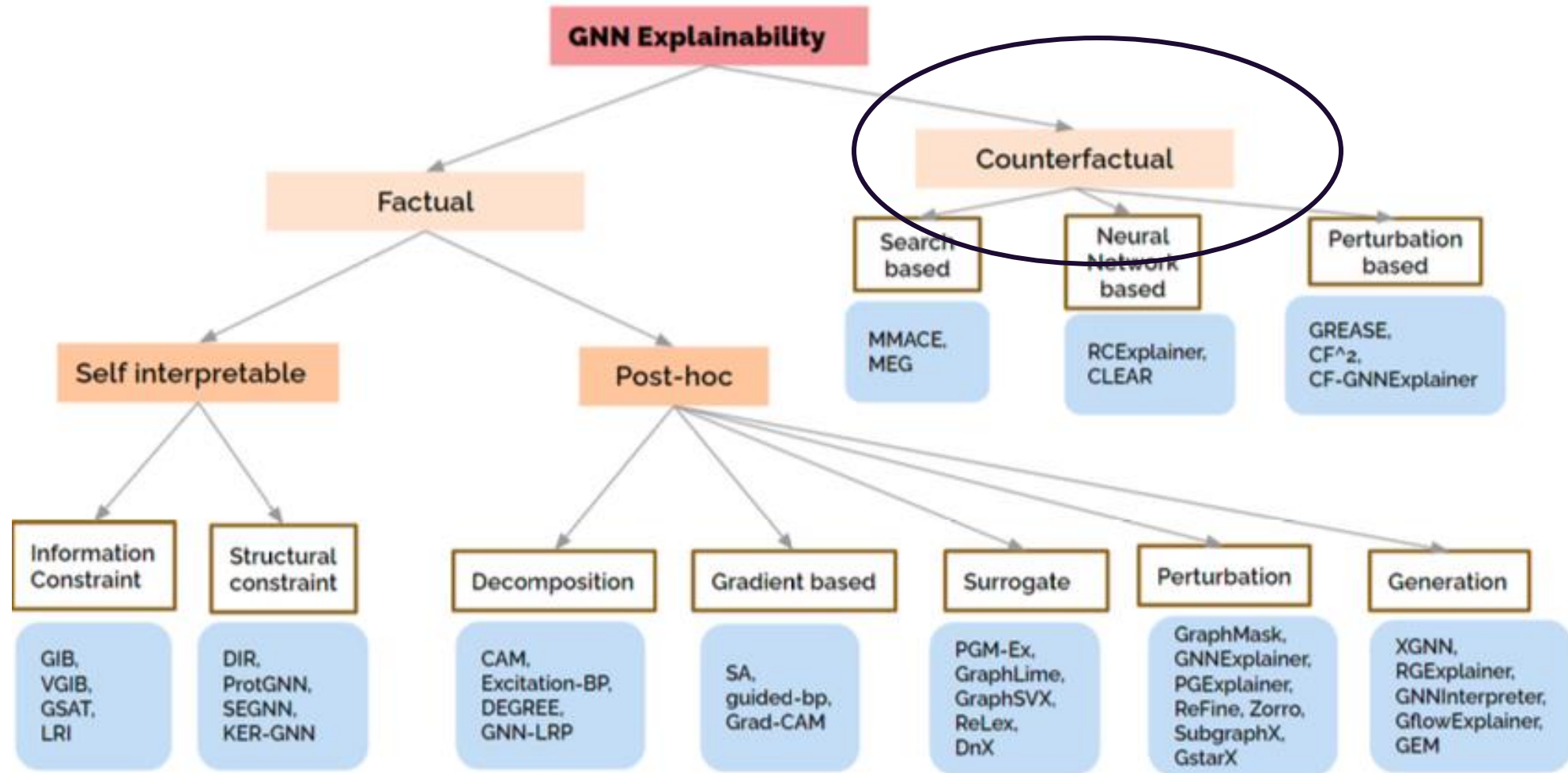
# Generation methods for explaining GNNs

- Apply generative methods to derive explanations
- Model level generative explanations
  - Generate sub-graph patterns that maximize predictions for a certain class
  - The sub-graphs are the explanations
- Instance level (particular graph / node classification) explanations
  - Generate a sub-graph conditioned on the input graph that maintains the prediction (similar to perturbation methods but differ in how the sub-graph is generated)

| Method             | Subgraph Extraction Strategy            | Scoring function | Constraints              |
|--------------------|-----------------------------------------|------------------|--------------------------|
| GNNExplainer [115] | Continuous relaxation                   | MI               | Size                     |
| SubgraphX [122]    | Monte Carlo Tree Search                 | SV               | Size, connectivity       |
| GraphMask [82]     | Layer-wise parameterized edge selection | $L_0$ norm       | Prediction divergence    |
| PGExplainer [56]   | Parameterized edge selection            | MI               | Size and/or connectivity |
| Zorro [23]         | Greedy selection                        | Fidelity         | Threshold fidelity       |
| ReFine [100]       | Parameterized edge attribution          | MI               | Number of edges          |
| GstarX [123]       | Monte Carlo sampling                    | HN-value         | Size                     |



# GNN XAI method taxonomy





# Counterfactual explanations on GNNs

- Find the minimum changes (counterfactual) to the input graph that **change** the model prediction
- Types of counterfactual methods include:
  - Perturbation
  - Neural network
  - Search
- See, <https://arxiv.org/abs/2306.01958>, for review



# XAI in PyG





# PyG Explainer

- Assume we have a trained GNN model
- Assume we've selected an explanation algorithm
- PyG Explainer is a wrapper class that implements the algorithm
  - Forward method returns and Explanation object
  - Consists of node and edge masks (the explanation)

```
from torch_geometric.data import Data
from torch_geometric.explain import Explainer, GNNExplainer

data = Data(...) # A homogeneous graph data object.

explainer = Explainer(
    model=model,
    algorithm=GNNExplainer(epochs=200),
    explanation_type='model',
    node_mask_type='attributes',
    edge_mask_type='object',
    model_config=dict(
        mode='multiclass_classification',
        task_level='node',
        return_type='log_probs', # Model returns log probabilities.
    ),
)

# Generate explanation for the node at index `10`:
explanation = explainer(data.x, data.edge_index, index=10)
print(explanation.edge_mask)
print(explanation.node_mask)
```



# PyG Explainer Algorithms

Several built in explanation algorithms

```
from torch_geometric.data import Data
from torch_geometric.explain import Explainer, GNNExplainer

data = Data(...) # A homogeneous graph data object.

explainer = Explainer(
    model=model,
    algorithm=GNNExplainer(epochs=200),
    explanation_type='model',
    node_mask_type='attributes',
    edge_mask_type='object',
    model_config=dict(
        mode='multiclass_classification',
        task_level='node',
        return_type='log_probs', # Model returns log probabilities.
    ),
)

# Generate explanation for the node at index `10`:
explanation = explainer(data.x, data.edge_index, index=10)
print(explanation.edge_mask)
print(explanation.node_mask)
```

|                    |                                                                                                                                                                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ExplainerAlgorithm | An abstract base class for implementing explainer algorithm                                                                                                                                                                                                           |
| DummyExplainer     | A dummy explainer that returns random explanations (use purposes).                                                                                                                                                                                                    |
| GNNExplainer       | The GNN-Explainer model from the " <a href="#">GNNExplainer: Generating Explanations for Graph Neural Networks</a> " paper for identifying compact subgraph structures and node features that play a crucial role in the predictions made by a GNN.                   |
| CaptumExplainer    | A <a href="#">Captum</a> -based explainer for identifying compact subgraph structures and node features that play a crucial role in the predictions made by a GNN.                                                                                                    |
| PGExplainer        | The PGExplainer model from the " <a href="#">Parameterized Explainer for Graph Neural Networks</a> " paper.                                                                                                                                                           |
| AttentionExplainer | An explainer that uses the attention coefficients produced by a GNN (e.g., <a href="#">GATConv</a> , <a href="#">GATv2Conv</a> , or <a href="#">TransformerConv</a> ) as explanations.                                                                                |
| GraphMaskExplainer | The GraphMask-Explainer model from the " <a href="#">Interpreting Graph Neural Networks for NLP With Differentiable Edge Masking</a> " paper for identifying compact subgraph structures and node features that play a crucial role in the predictions made by a GNN. |



# PyG Explanation Metrics

Several built in metrics

## Explanation Metrics

The quality of an explanation can be judged by a variety of different methods. PyG supports the following metrics out-of-the-box:

`groundtruth_metrics`

Compares and evaluates an explanation mask with the ground-truth explanation mask.

`fidelity`

Evaluates the fidelity of an `Explainer` given an `Explanation`, as described in the ["GraphFramEx: Towards Systematic Evaluation of Explainability Methods for Graph Neural Networks"](#) paper.

`characterization_score`

Returns the componentwise characterization score as described in the ["GraphFramEx: Towards Systematic Evaluation of Explainability Methods for Graph Neural Networks"](#) paper.

`fidelity_curve_auc`

Returns the AUC for the fidelity curve as described in the ["GraphFramEx: Towards Systematic Evaluation of Explainability Methods for Graph Neural Networks"](#) paper.

`unfaithfulness`

Evaluates how faithful an `Explanation` is to an underlying GNN predictor, as described in the ["Evaluating Explainability for Graph Neural Networks"](#) paper.



## Python XAI resources

- SHAP - <https://shap.readthedocs.io/en/latest/>
- ELI5 - <https://eli5.readthedocs.io/en/latest/>
- Captum - <https://captum.ai/> (deep learning XAI library)
- Interpretable Machine Learning by Molnar (free online text)  
<https://christophm.github.io/interpretable-ml-book/>

